Intelligent Systems,
MIPT

# Natural Language Processing

L6 / 17.03.26

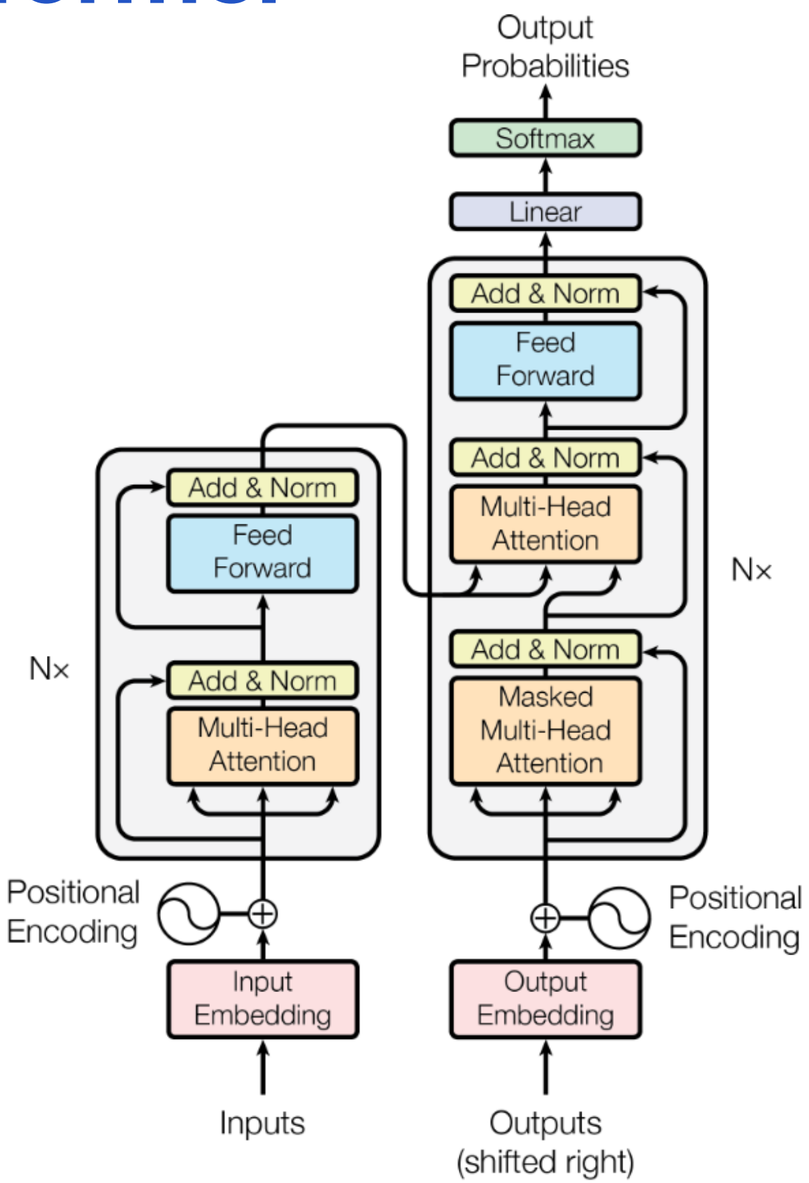
- Course page
- GitHub

# Content

## Lecture 6: How modern LLMs evolved beyond the original Transformer

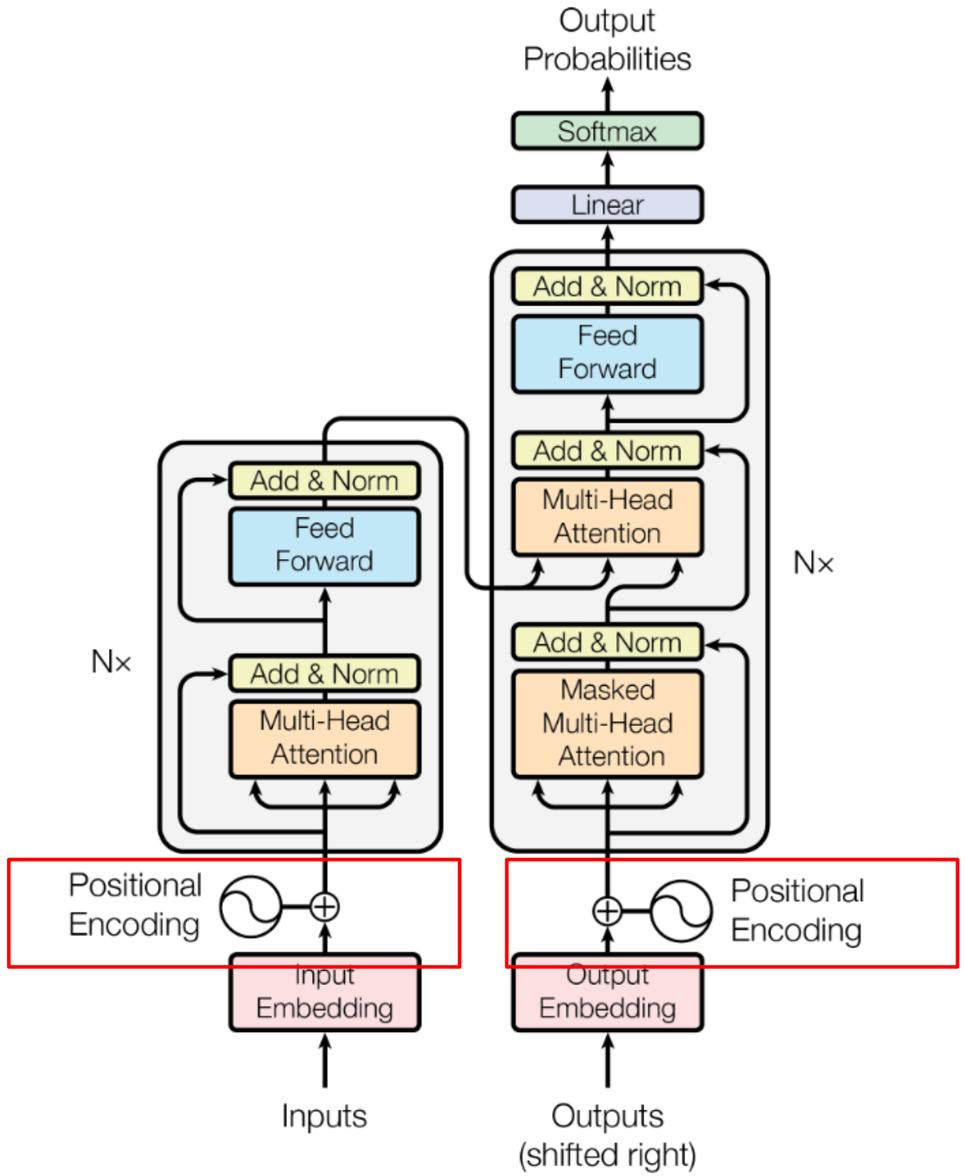
- Positional embeddings to RoPE and ALiBi
- KV cache
- MHA, MQA, GQA, MLA
- RMSNorm and SwiGLU

# Vanilla Transformer

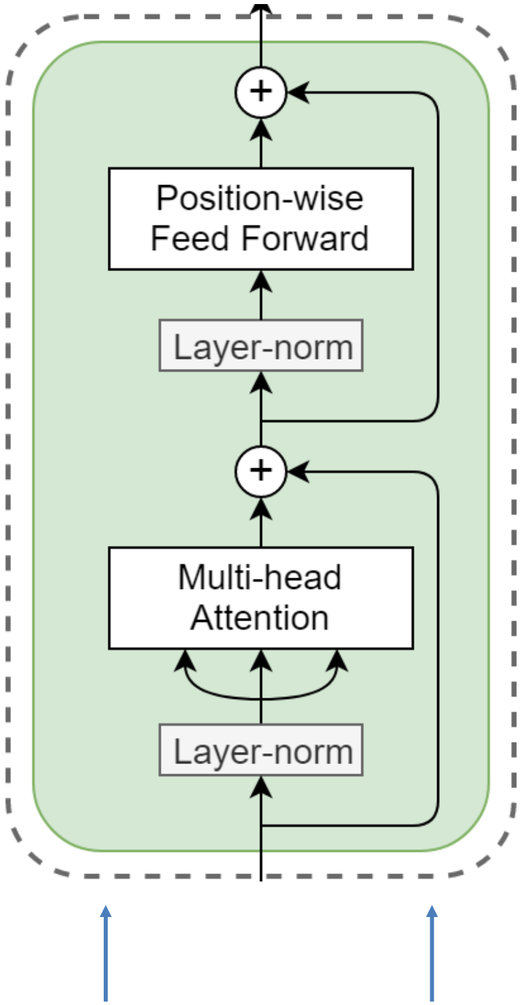


# Vanilla Transformer: PE

🤔 Could we do better?



# PE: motivation




The dog bit the man == The man bit the dog

# Positional Embeddings

- $x = (x_1, \ldots, x_n)$ token vectors of the input sequence
- $z = (z_1, \ldots, z_n)$ self-attention head output vectors
- $d_x$, $d_z$ the dimensions of vectors x and z, respectively
- $Q$, $K$, $V$ vectors of queries, keys, and values for the header layer
- $W^Q$, $W^K$, $W^V$ weight matrices for generating query vectors, keys, and values for the head layer

# Positional Embeddings

- $x = (x_1, \ldots, x_n)$ token vectors of the input sequence
- $z = (z_1, \ldots, z_n)$ self-attention head output vectors
- $d_x$, $d_z$ the dimensions of vectors x and z, respectively
- $Q$, $K$, $V$ vectors of queries, keys, and values for the header layer
- $W^Q$, $W^K$, $W^V$ weight matrices for generating query vectors, keys, and values for the head layer

- Formulas for calculating self-attention:

$$e_{ij} = \frac{\left(x_i W^Q\right)\left(x_j W^K\right)^T}{\sqrt{d_z}}, \qquad z_i = \sum_{j=1}^{n} \alpha_{ij}\left(x_{ij} W^V\right) \qquad \alpha_{ij} = \mathrm{softmax}(e)_{ij} = \frac{\exp(e_{ij})}{\sum_{k=1}^{n} \exp(e_{ik})}$$

# Absolute positional embeddings

- Sinusoidal and Trainable Absolute [2017]
- Each absolute position is represented by a vector $\hat{p}_t \in \mathbb{R}^{d_x}$

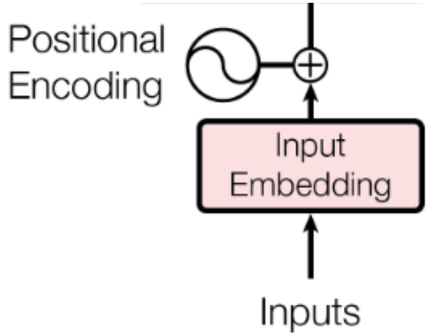
$$\hat{p}_t^i = \begin{cases} \sin(w_k t), & i = 2k \\ \cos(w_k t), & i = 2k+1 \end{cases}, \qquad w_k = \frac{1}{10000^{2k/d_x}}$$

# Absolute positional embeddings

- Sinusoidal and Trainable Absolute [2017]
- Each absolute position is represented by a vector $\hat{p}_t \in \mathbb{R}^{d_x}$

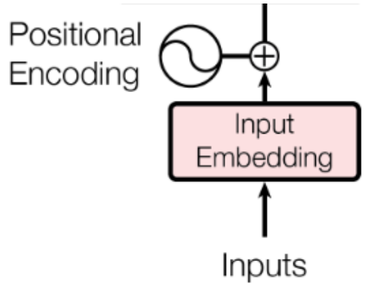
$$\hat{p}_t^i = \begin{cases} \sin(w_k t), & i = 2k \\ \cos(w_k t), & i = 2k+1 \end{cases}, \qquad w_k = \frac{1}{10000^{2k/d_x}}$$

- The position vector is added to the token vector at the model input



# Absolute positional embeddings

- The position vector is added to the token vector at the model input

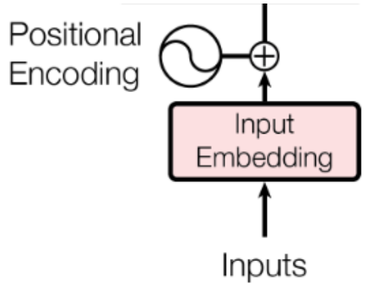



- **Drawbacks:**

  > No relative information between tokens

# Absolute positional embeddings

- The position vector is added to the token vector at the model input



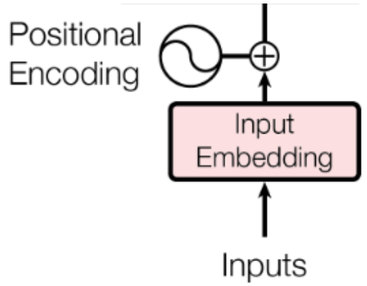

- **Drawbacks:**

  > No relative information between tokens

  > No extrapolation -> performance declines for positions with longer training

# Absolute positional embeddings

- The position vector is added to the token vector at the model input




- **Drawbacks:**

  > No relative information between tokens

  > No extrapolation -> performance declines for positions with longer training

  > This increases the number of parameters

# Relative positional embeddings

- Not a single index is encoded, rather a pair of positions at different distances from one another
- Performed by modifying the attention calculation

# Relative positional embeddings

- Not a single index is encoded, rather a pair of positions at different distances from one another
- Performed by modifying the attention calculation
- For each distance $i - j$, two vectors are trained: $a_{ij}^K, a_{ij}^V \in \mathbb{R}^{d_z}$
- So, attention calculation:

$$e_{ij} = \frac{\left(x_i W^Q\right)\left(x_j W^K + a_{ij}^K\right)^T}{\sqrt{d_z}}, \qquad z_i = \sum_{j=1}^{n} \alpha_{ij}\left(x_{ij} W^V + a_{ij}^V\right)$$

# Relative positional embeddings

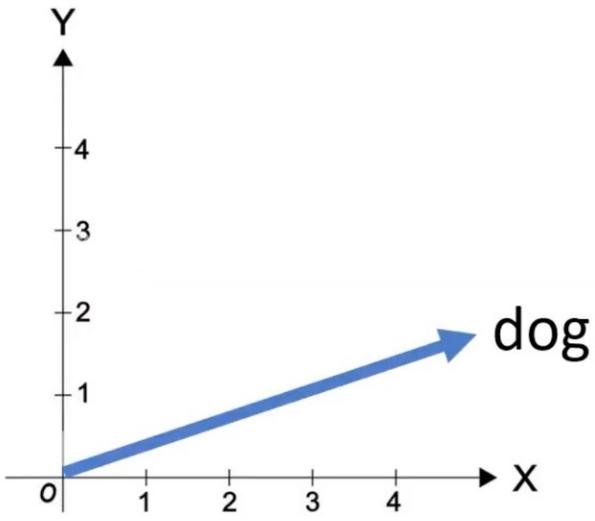
- Not a single index is encoded, rather a pair of positions at different distances from one another
- Performed by modifying the attention calculation
- For each distance $i - j$, two vectors are trained: $a_{ij}^K, a_{ij}^V \in \mathbb{R}^{d_z}$
- So, attention calculation:

$$e_{ij} = \frac{\left(x_i W^Q\right)\left(x_j W^K + a_{ij}^K\right)^T}{\sqrt{d_z}}, \qquad z_i = \sum_{j=1}^{n} \alpha_{ij}\left(x_{ij} W^V + a_{ij}^V\right)$$

Examples: Relative Position Repr., Transformer-XL, T5, DeBERTa

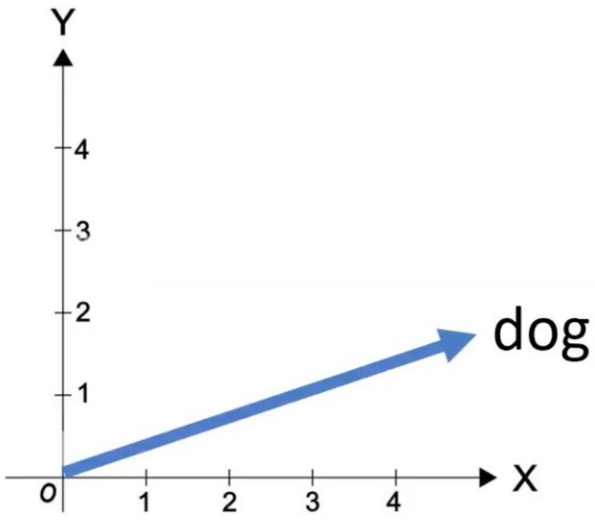
# RoPE (Rotary PE): idea

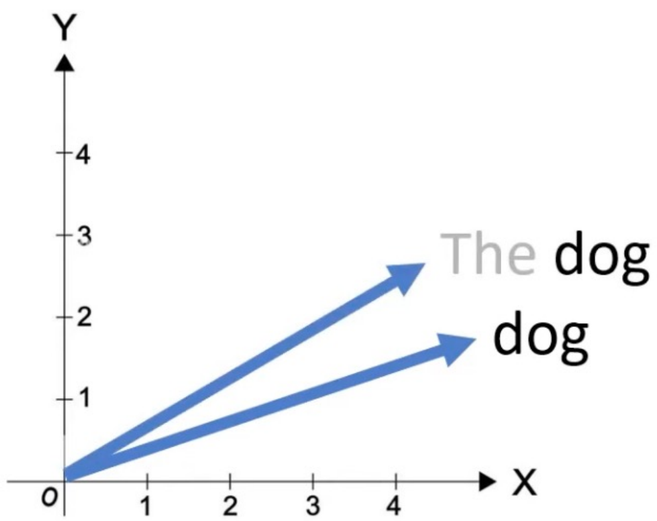
- Want to keep absolute positions idea?



# RoPE (Rotary PE): idea

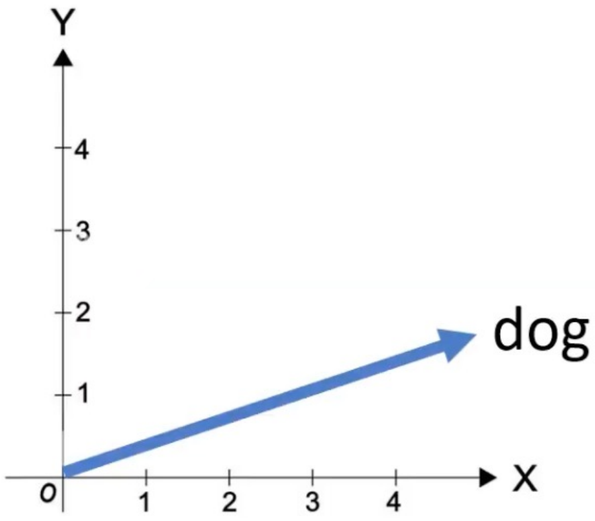
- Want to keep absolute positions idea?

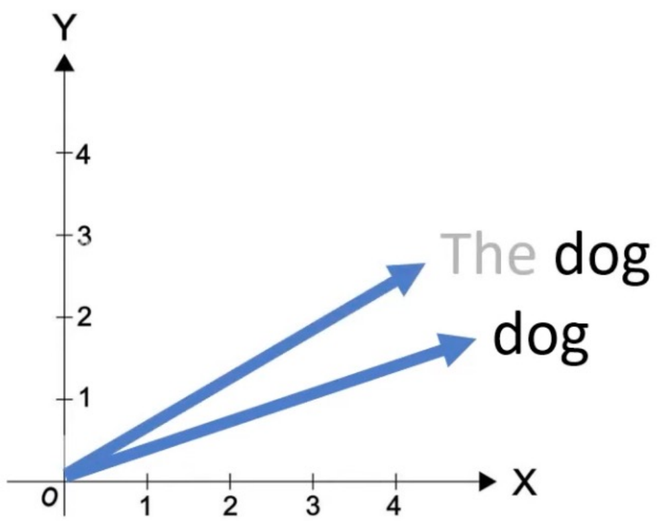


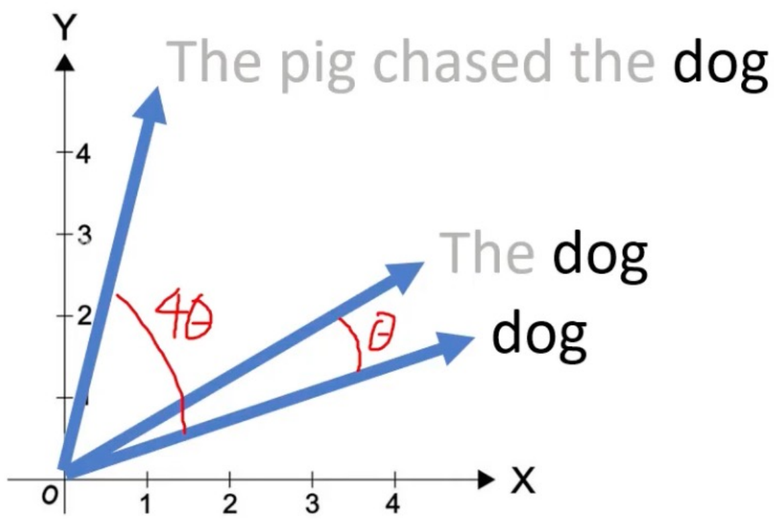



# RoPE (Rotary PE): idea

- Want to keep absolute positions idea?

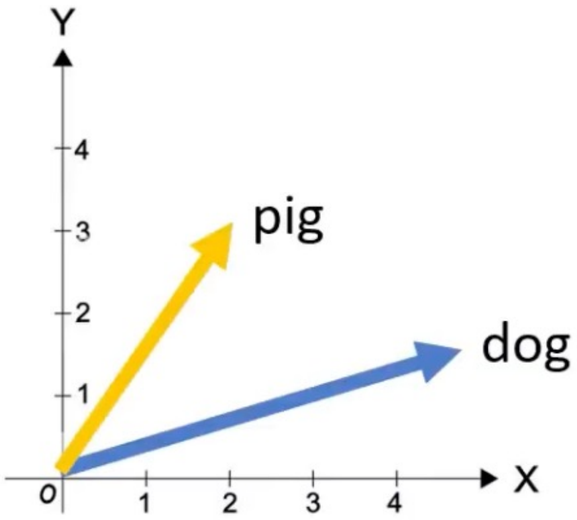


# RoPE (Rotary PE): idea

- Want to keep ~~absolute~~ relative positions idea?

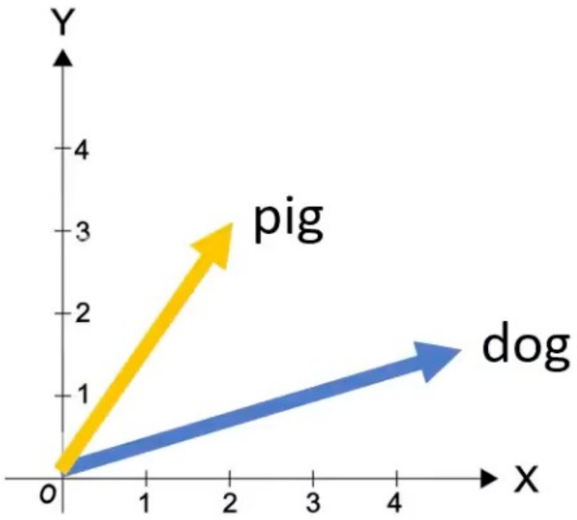



The pig chased the dog

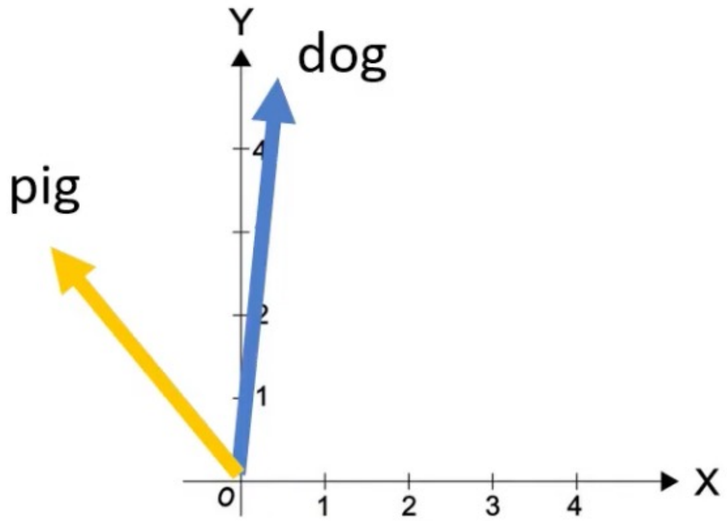
RoPE: explanation video

# RoPE (Rotary PE): idea

- Want to keep ~~absolute~~ relative positions idea?




The pig chased the dog




Once upon a time, the pig chased the dog

RoPE: explanation video

# RoPE (Rotary PE)

- Let q and k be query and key vectors respectively and let m and n be the absolute positions of the corresponding tokens

$$\langle f_q(\boldsymbol{x}_m, m), f_k(\boldsymbol{x}_n, n)\rangle = g(\boldsymbol{x}_m, \boldsymbol{x}_n, m-n).$$

$$\boldsymbol{q}_m^\intercal \boldsymbol{k}_n = \langle f_q(\boldsymbol{x}_m, m), f_k(\boldsymbol{x}_n, n)\rangle = g(\boldsymbol{x}_m, \boldsymbol{x}_n, n-m),$$

# RoPE (Rotary PE)

- Let q and k be query and key vectors respectively and let m and n be the absolute positions of the corresponding tokens

$$\langle f_q(\boldsymbol{x}_m, m), f_k(\boldsymbol{x}_n, n)\rangle = g(\boldsymbol{x}_m, \boldsymbol{x}_n, m-n).$$

$$\boldsymbol{q}_m^\intercal \boldsymbol{k}_n = \langle f_q(\boldsymbol{x}_m, m), f_k(\boldsymbol{x}_n, n)\rangle = g(\boldsymbol{x}_m, \boldsymbol{x}_n, n-m),$$

- 2D-case:

$$\begin{aligned} f_q(\boldsymbol{x}_m, m) &= (\boldsymbol{W}_q \boldsymbol{x}_m) e^{im\theta} \\ f_k(\boldsymbol{x}_n, n) &= (\boldsymbol{W}_k \boldsymbol{x}_n) e^{in\theta} \\ g(\boldsymbol{x}_m, \boldsymbol{x}_n, m-n) &= \mathrm{Re}[(\boldsymbol{W}_q \boldsymbol{x}_m)(\boldsymbol{W}_k \boldsymbol{x}_n)^* e^{i(m-n)\theta}] \end{aligned}$$

# RoPE (Rotary PE)

- Let q and k be query and key vectors respectively and let m and n be the absolute positions of the corresponding tokens

$$\langle f_q(\boldsymbol{x}_m, m), f_k(\boldsymbol{x}_n, n)\rangle = g(\boldsymbol{x}_m, \boldsymbol{x}_n, m-n).$$

$$\boldsymbol{q}_m^\intercal \boldsymbol{k}_n = \langle f_q(\boldsymbol{x}_m, m), f_k(\boldsymbol{x}_n, n)\rangle = g(\boldsymbol{x}_m, \boldsymbol{x}_n, n-m),$$

- 2D-case:

$$\begin{aligned} f_q(\boldsymbol{x}_m, m) &= (\boldsymbol{W}_q \boldsymbol{x}_m) e^{im\theta} \\ f_k(\boldsymbol{x}_n, n) &= (\boldsymbol{W}_k \boldsymbol{x}_n) e^{in\theta} \\ g(\boldsymbol{x}_m, \boldsymbol{x}_n, m-n) &= \mathrm{Re}[(\boldsymbol{W}_q \boldsymbol{x}_m)(\boldsymbol{W}_k \boldsymbol{x}_n)^* e^{i(m-n)\theta}] \end{aligned}$$

$$f_{\{q,k\}}(\boldsymbol{x}_m, m) = \begin{pmatrix} \cos m\theta & -\sin m\theta \\ \sin m\theta & \cos m\theta \end{pmatrix} \begin{pmatrix} W_{\{q,k\}}^{(11)} & W_{\{q,k\}}^{(12)} \\ W_{\{q,k\}}^{(21)} & W_{\{q,k\}}^{(22)} \end{pmatrix} \begin{pmatrix} x_m^{(1)} \\ x_m^{(2)} \end{pmatrix}$$

# RoPE (Rotary PE)

- General case: $f_{\{q,k\}}(\boldsymbol{x}_m, m) = \boldsymbol{R}^d_{\Theta,m} \boldsymbol{W}_{\{q,k\}} \boldsymbol{x}_m$

$$\boldsymbol{R}^d_{\Theta,m} = \begin{pmatrix} \cos m\theta_1 & -\sin m\theta_1 & 0 & 0 & \cdots & 0 & 0 \\ \sin m\theta_1 & \cos m\theta_1 & 0 & 0 & \cdots & 0 & 0 \\ 0 & 0 & \cos m\theta_2 & -\sin m\theta_2 & \cdots & 0 & 0 \\ 0 & 0 & \sin m\theta_2 & \cos m\theta_2 & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \cdots & \cos m\theta_{d/2} & -\sin m\theta_{d/2} \\ 0 & 0 & 0 & 0 & \cdots & \sin m\theta_{d/2} & \cos m\theta_{d/2} \end{pmatrix}$$

$$\Theta = \{\theta_i = 10000^{-2(i-1)/d}, i \in [1, 2, ..., d/2]\}$$

- The RoPE method involves rotating the query/key vector by an angle that depends on its position index

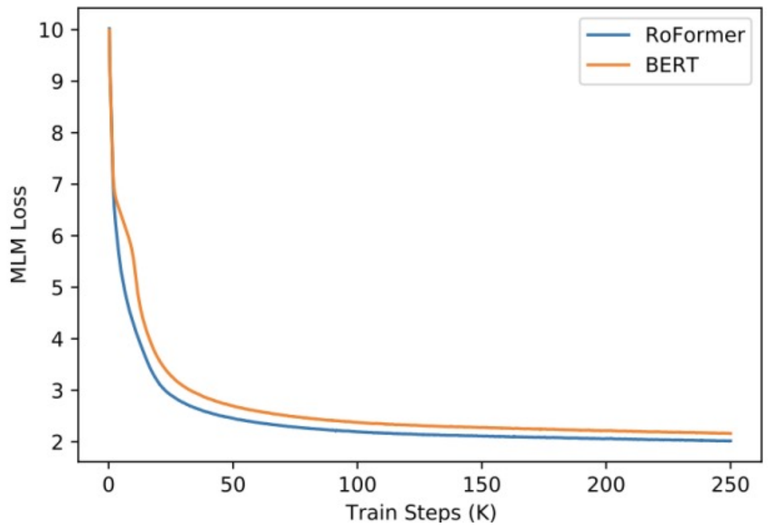
# RoPE (Rotary PE)

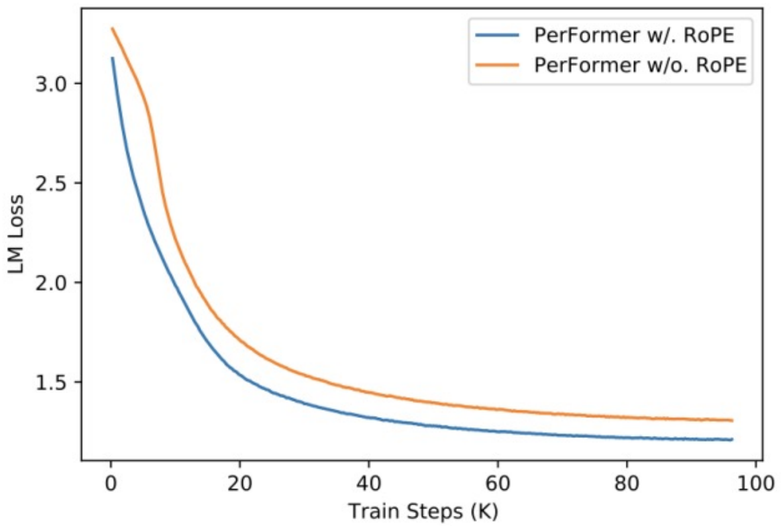
- **Pros:**

  > One of the most popular approaches to positional encoding, used in LLaMA, Qwen and Mistral

  > Unlike Absolute, it is applied not to individual coordinates but to pairs, and uses multiplication by sin/cos instead of addition

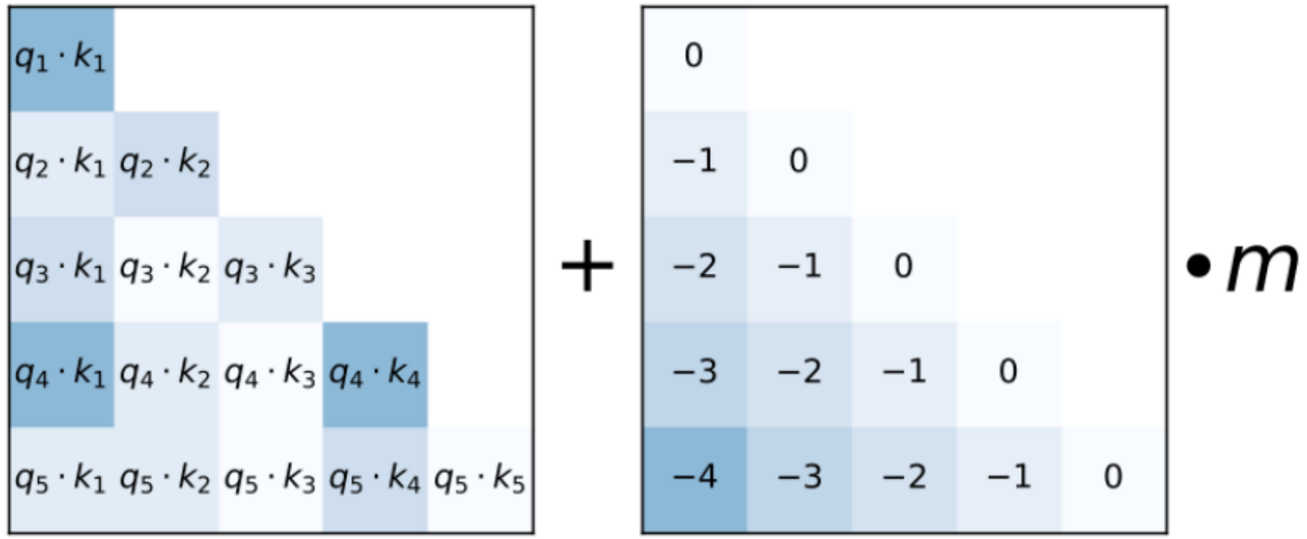
  > Used as a foundation  for more advanced methods



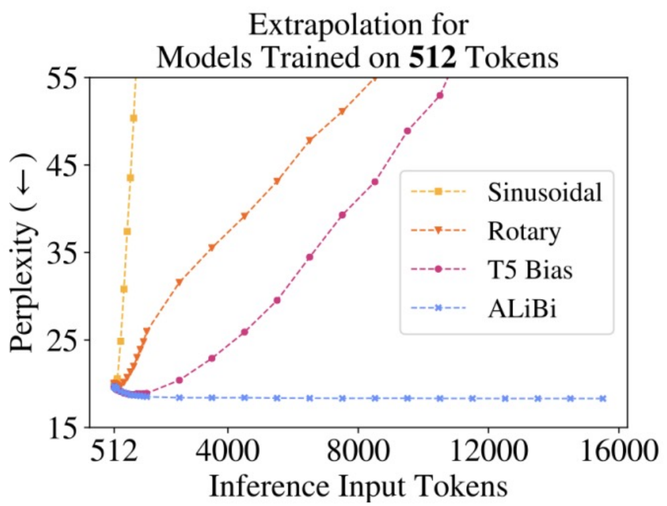
# ALiBi

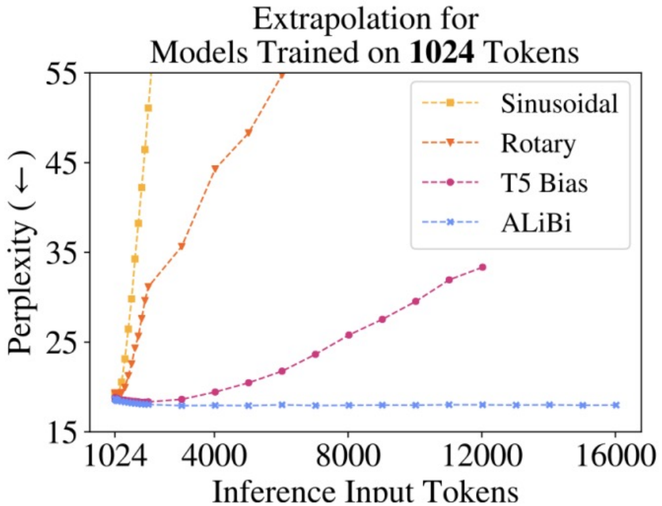
- Instead of adding positional embeddings or reparameterising the attention calculation, a scalar is added to $e_{ij}$
- Scalars are not trainable and are defined as the product $m(i - j)$, where $m$ is a number set at the start, unique to each head

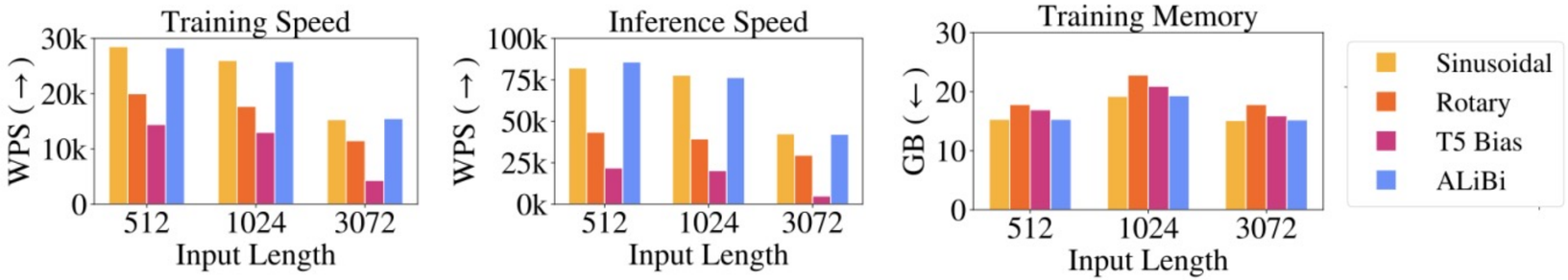


# ALiBi

- ALiBi proves to be quite effective in terms of speed and memory and provides a higher level of generalisation, enabling models to handle contexts that are significantly longer than those used during training




ALiBi paper

# Another way for long context

- RoPE is trained at fixed frequencies $\theta_k$ → for lengths > $N_{train}$, high-frequency components fall outside the distribution

# Another way for long context

- **RoPE is trained at fixed frequencies $\theta_k$ → for lengths > $N_{train}$, high-frequency components fall outside the distribution**

- **Suggested:**

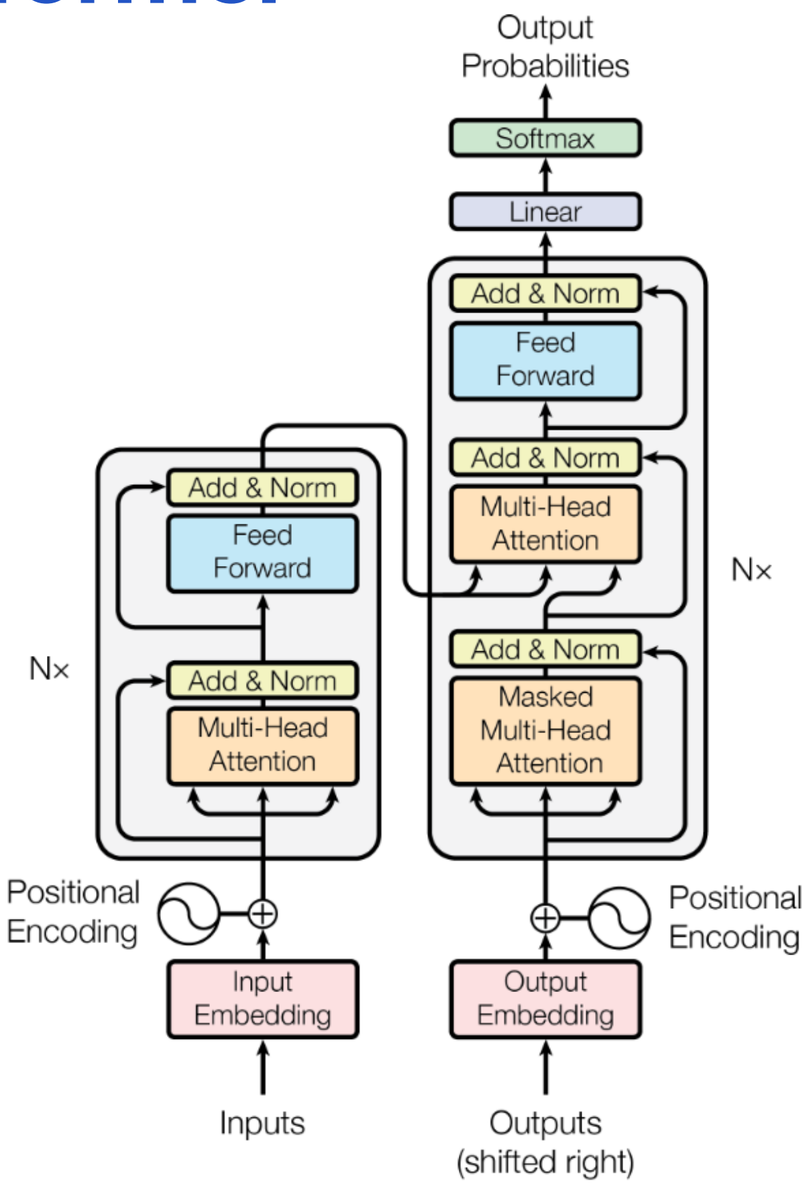
  **> Position Interpolation -** linear scaling of positions

  **> NTK-aware Scaling -** stretch the base period $\theta$ whilst leaving the high-frequency components unchanged

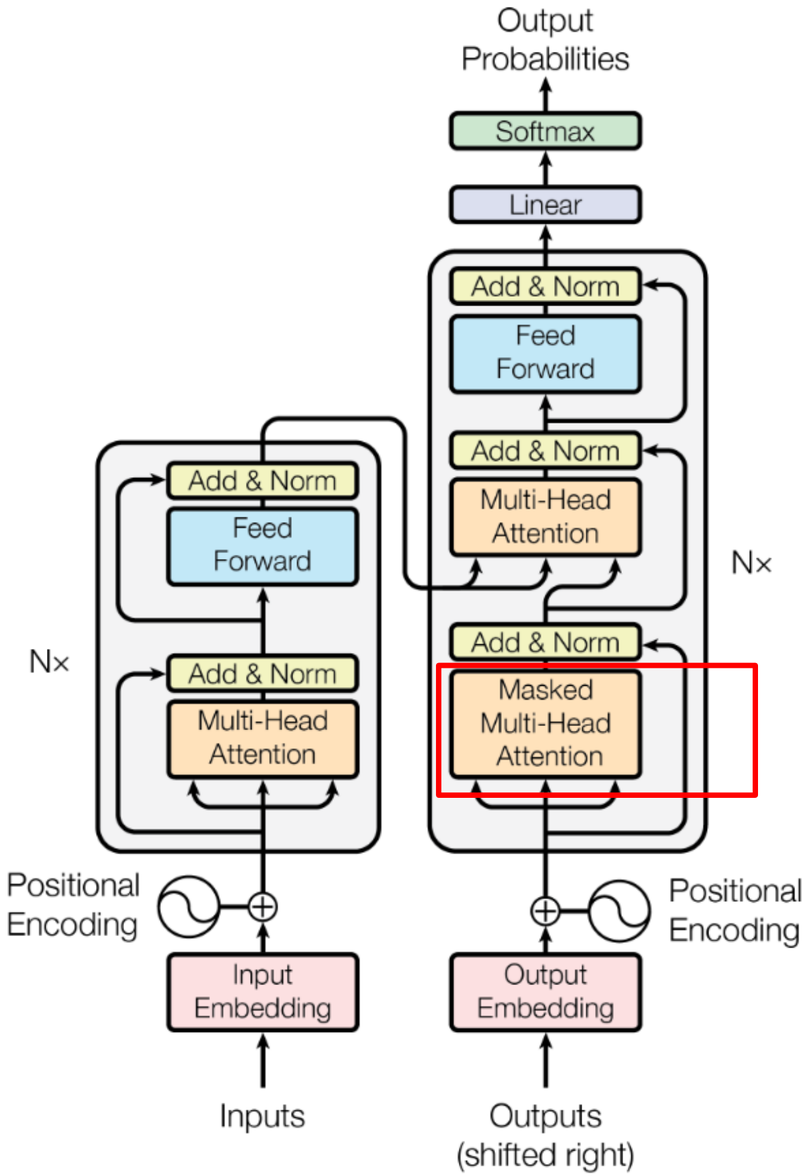
  **> YaRN** - NTK + attention scaling + temperature.

  And more more, like **Sliding Window Attention**, **Ring Attention**, **Continual Pre-training**

# Vanilla Transformer



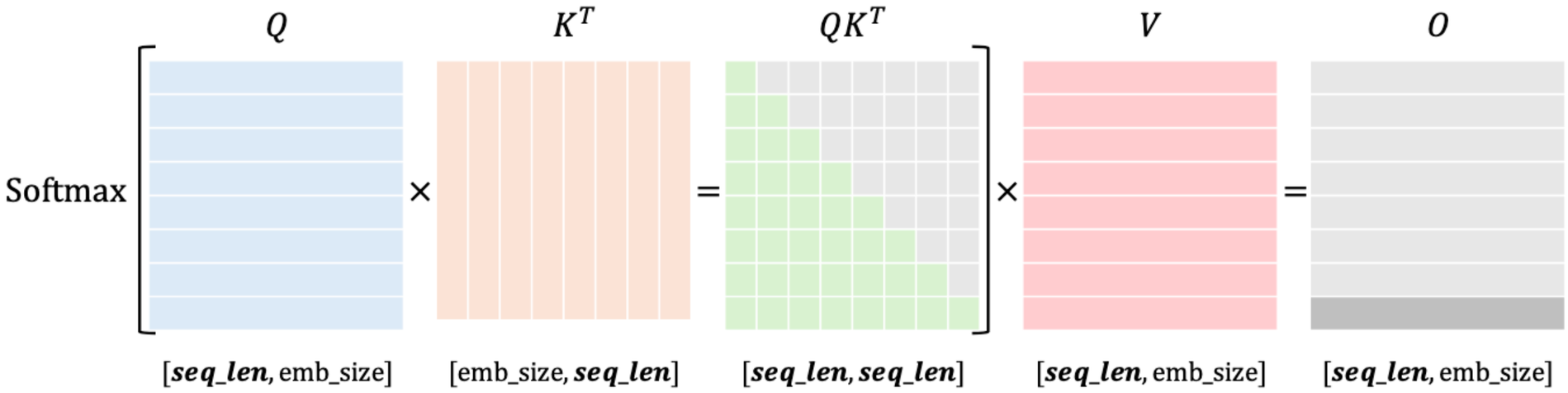
# Vanilla Transformer: Attention



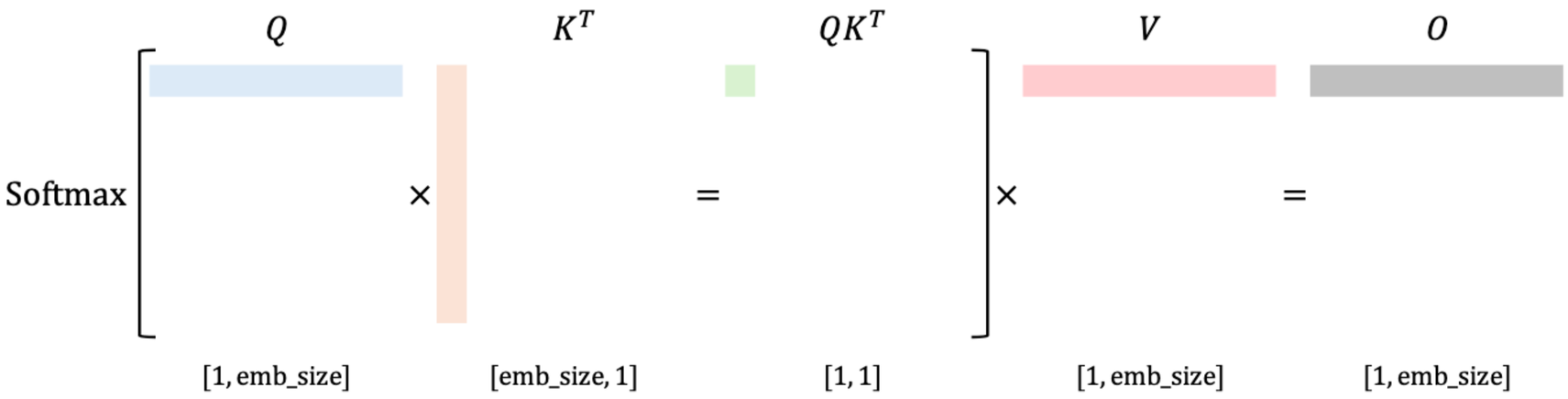

🤔 Could we do better?

# KV-cache: intuition

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right)\mathbf{V}$$

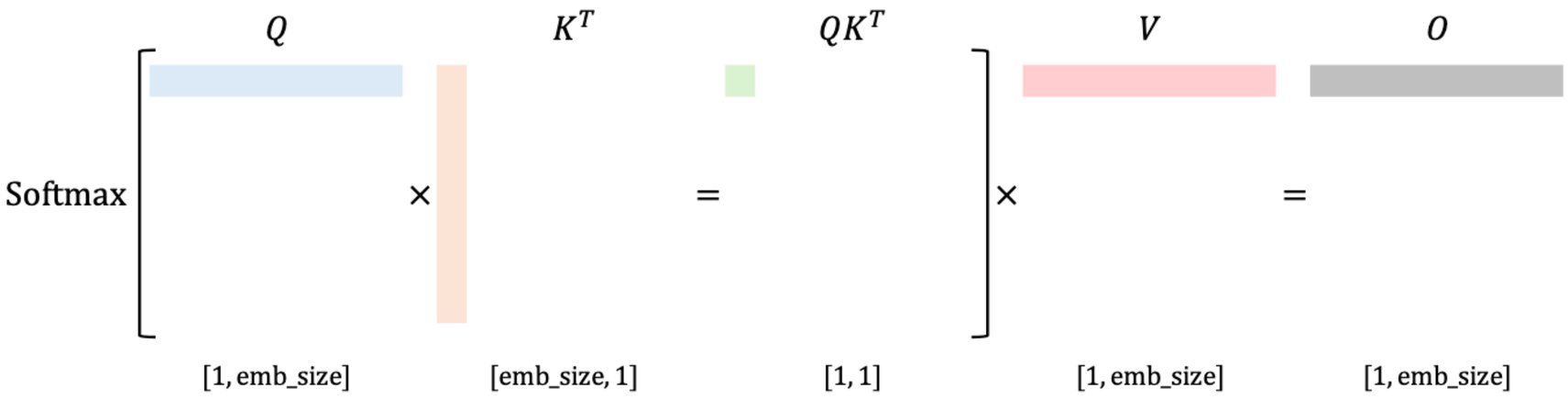


# KV-cache: intuition

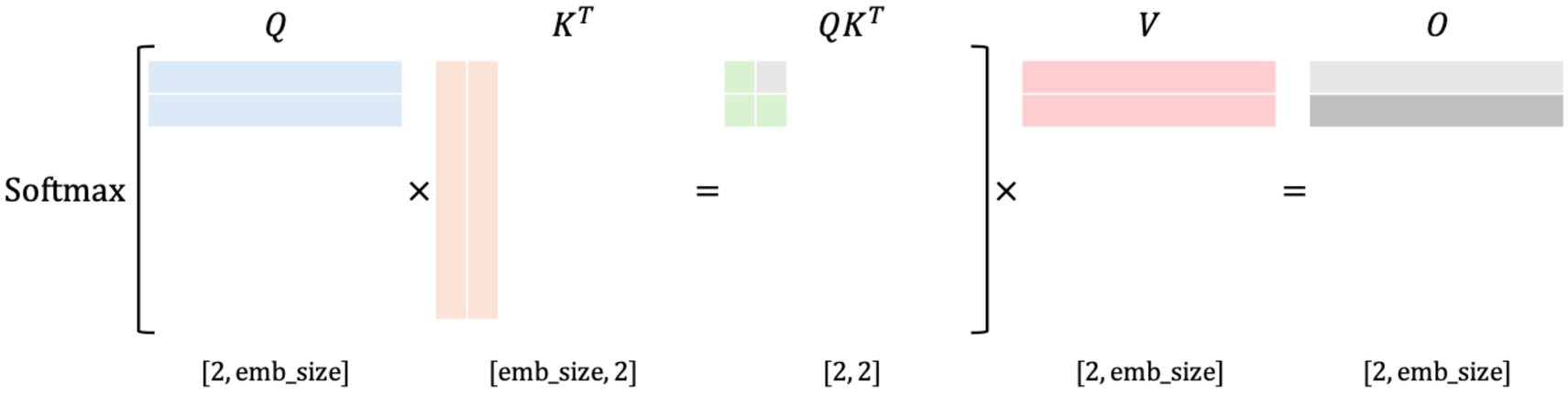



<bos> Transformers

# KV-cache: intuition

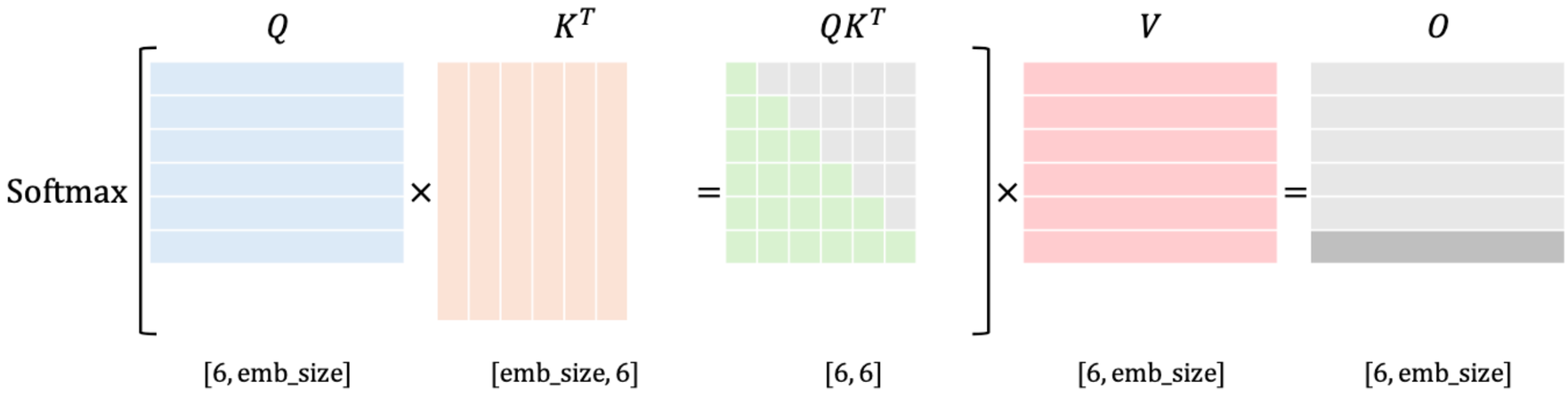



<bos> Transformers



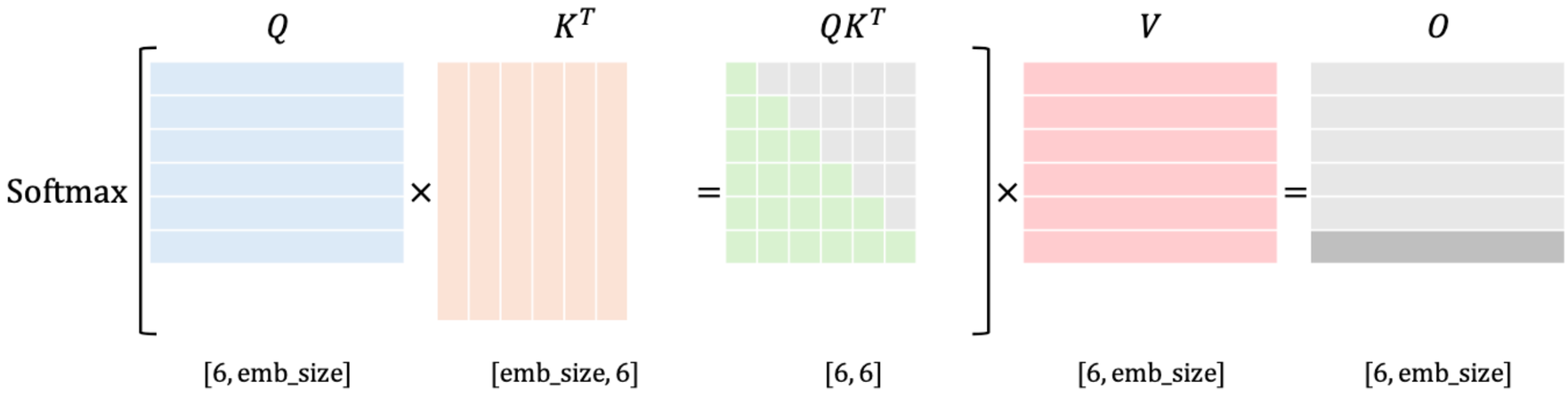

<bos> Transformers predict

# KV-cache: intuition

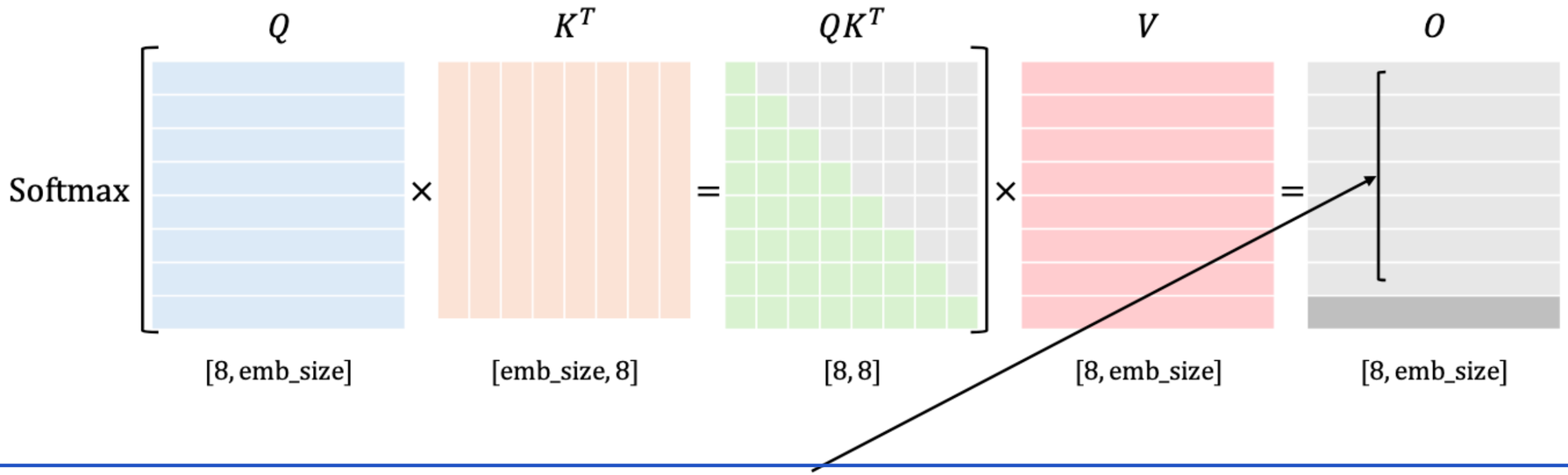



<bos> Transformers predict tomorrow's words from today's

# KV-cache: intuition

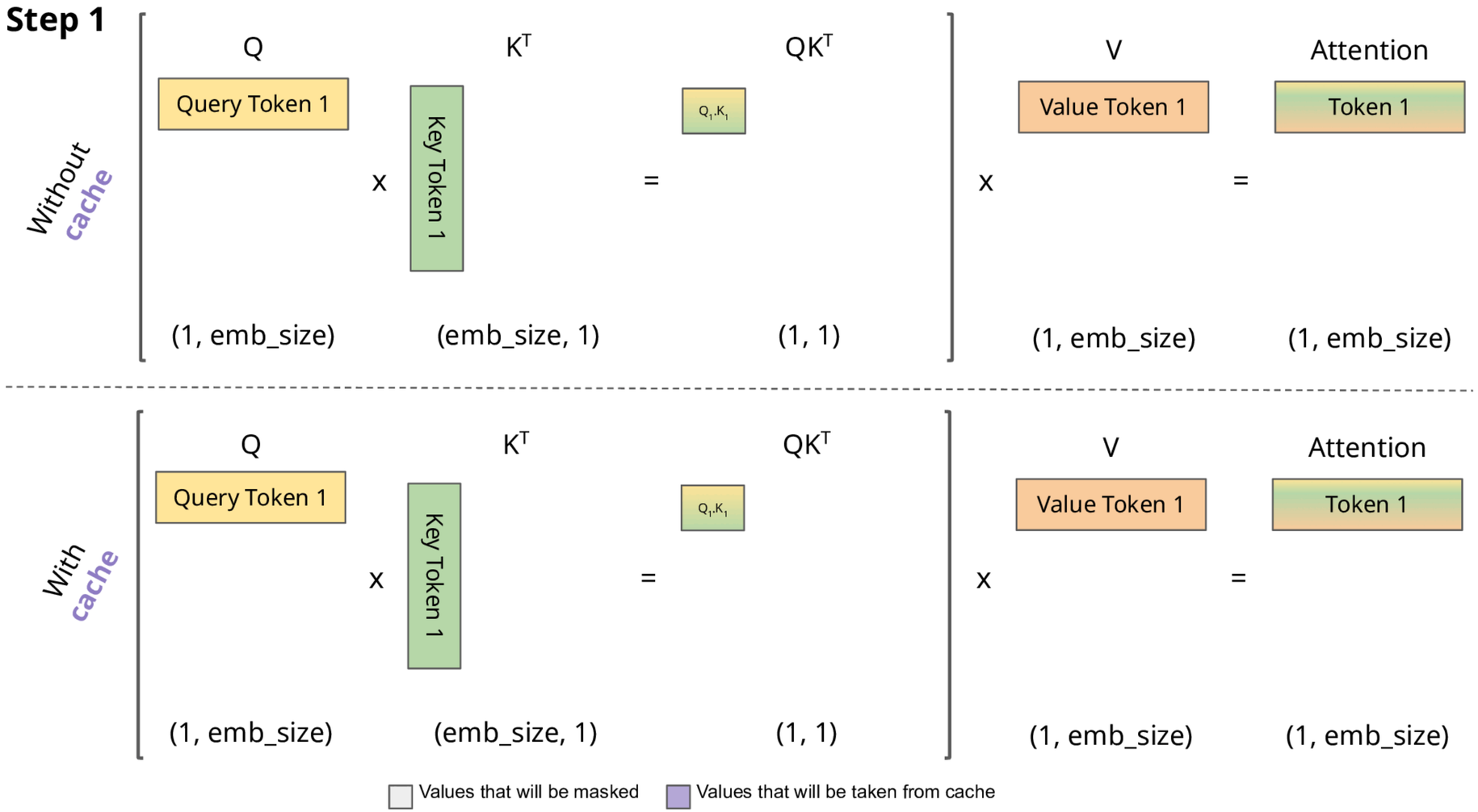



<bos> Transformers predict tomorrow's words from today's




We don't use these vectors to generate the next token!

# KV-cache


Step 1
Without cache
Q
Query Token 1
x
$K^T$
Key Token 1
=
$QK^T$
$Q_1 \cdot K_1$
V
Value Token 1
x
=
Attention
Token 1
(1, emb_size)
(emb_size, 1)
(1, 1)
(1, emb_size)
(1, emb_size)
With cache
Q
Query Token 1
x
$K^T$
Key Token 1
=
$QK^T$
$Q_1 \cdot K_1$
V
Value Token 1
x
=
Attention
Token 1
(1, emb_size)
(emb_size, 1)
(1, 1)
(1, emb_size)
(1, emb_size)
Values that will be masked
Values that will be taken from cache

# KV-cache

- Cache the past keys and values once → reuse them for all future queries:

$$\mathbf{K}_{\text{cache}} = [k_1, k_2, \ldots, k_{t-1}]^T, \quad \mathbf{V}_{\text{cache}} = [v_1, v_2, \ldots, v_{t-1}]^T$$

$$o_t = \text{Softmax}\left(\frac{q_t^T \cdot [\mathbf{K}_{\text{cache}}^T, k_t]}{\sqrt{d_k}}\right)[\mathbf{V}_{\text{cache}}, v_t]^T$$

# KV-cache

- Cache the past keys and values once → reuse them for all future queries:

$$\mathbf{K}_{\text{cache}} = [k_1, k_2, \ldots, k_{t-1}]^T, \quad \mathbf{V}_{\text{cache}} = [v_1, v_2, \ldots, v_{t-1}]^T$$

$$o_t = \text{Softmax}\left(\frac{q_t^T \cdot [\mathbf{K}_{\text{cache}}^T, k_t]}{\sqrt{d_k}}\right)[\mathbf{V}_{\text{cache}}, v_t]^T$$

- Complexity per step reduces

| With KV-Cache | Standard Inference | Speedup |
|---|---|---|
| 11.7 s | 1 min 1 s | ~5.21x times faster |

# KV-cache

- Cache the past keys and values once → reuse them for all future queries:

$$\mathbf{K}_{\text{cache}} = [k_1, k_2, \ldots, k_{t-1}]^T, \quad \mathbf{V}_{\text{cache}} = [v_1, v_2, \ldots, v_{t-1}]^T$$

$$o_t = \text{Softmax}\left(\frac{q_t^T \cdot [\mathbf{K}_{\text{cache}}^T, k_t]}{\sqrt{d_k}}\right)[\mathbf{V}_{\text{cache}}, v_t]^T$$

- Complexity per step reduces

| With KV-Cache | Standard Inference | Speedup |
|---|---|---|
| 11.7 s | 1 min 1 s | ~5.21x times faster |

- Size: $2 * precision * n_{layers} * d_{model} * seq_{len} * batch$

  Example: OPT-30B with $2 * 2 * 48 * 7168 * 1024 * 128 = 180\ GB$

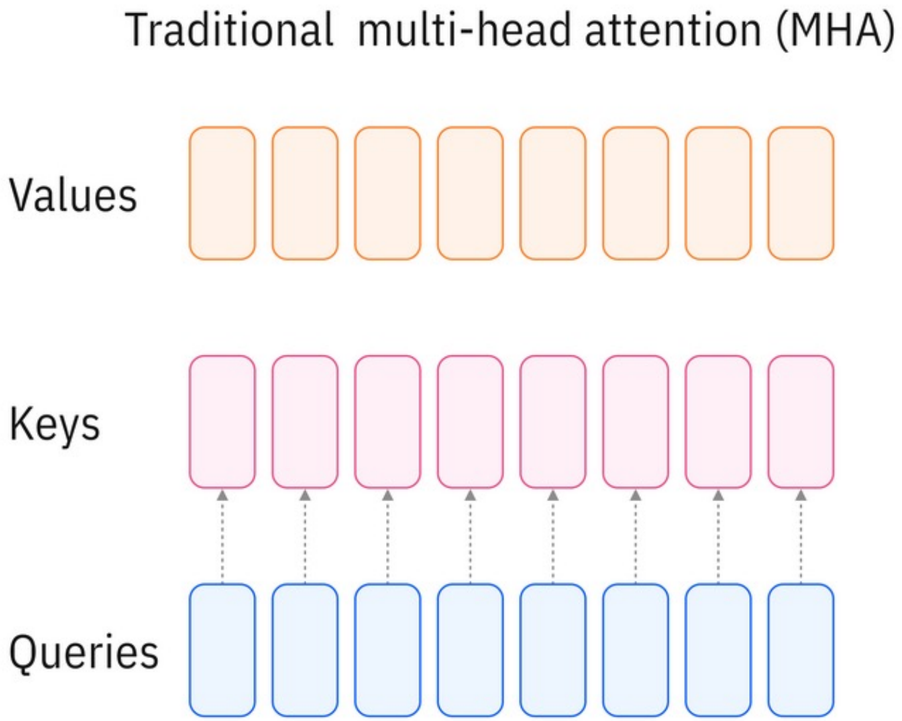
KV-cache HF paper

# Multi-Head Attention (MHA)

- MHA splits Q, K and V and into multiple heads, each with independent and , thereby increasing the model's capacity and flexibility

$$\mathrm{MHA}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \mathrm{Concat}(\mathrm{head}_1, \ldots, \mathrm{head}_H)\, W_O$$

$$\mathrm{head}_h = \mathrm{Attention}(\mathbf{Q}_h, \mathbf{K}_h, \mathbf{V}_h) = \mathrm{softmax}\left(\frac{\mathbf{Q}_h \mathbf{K}_h^\top}{\sqrt{d_{\mathrm{head}}}}\right) \mathbf{V}_h$$



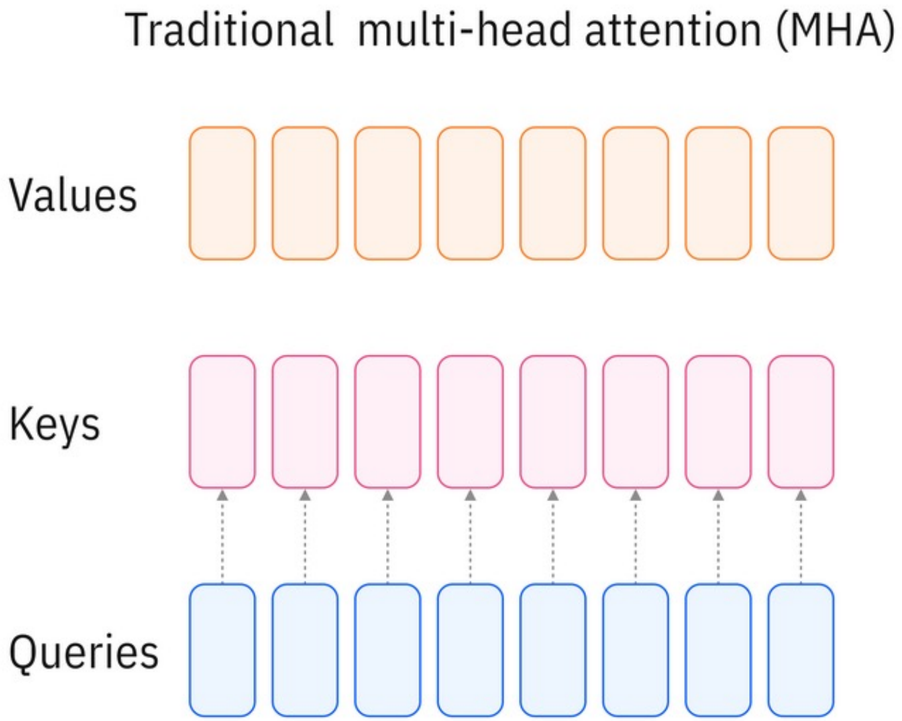
# Multi-Head Attention (MHA)

- MHA splits Q, K and V and into multiple heads, each with independent and , thereby increasing the model's capacity and flexibility

$$\mathrm{MHA}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \mathrm{Concat}(\mathrm{head}_1, \ldots, \mathrm{head}_H)\, W_O$$

$$\mathrm{head}_h = \mathrm{Attention}(\mathbf{Q}_h, \mathbf{K}_h, \mathbf{V}_h) = \mathrm{softmax}\left(\frac{\mathbf{Q}_h \mathbf{K}_h^\top}{\sqrt{d_{\mathrm{head}}}}\right) \mathbf{V}_h$$

- **Pros:** capturing diverse features, each attention head can learn different representations, parallel computation



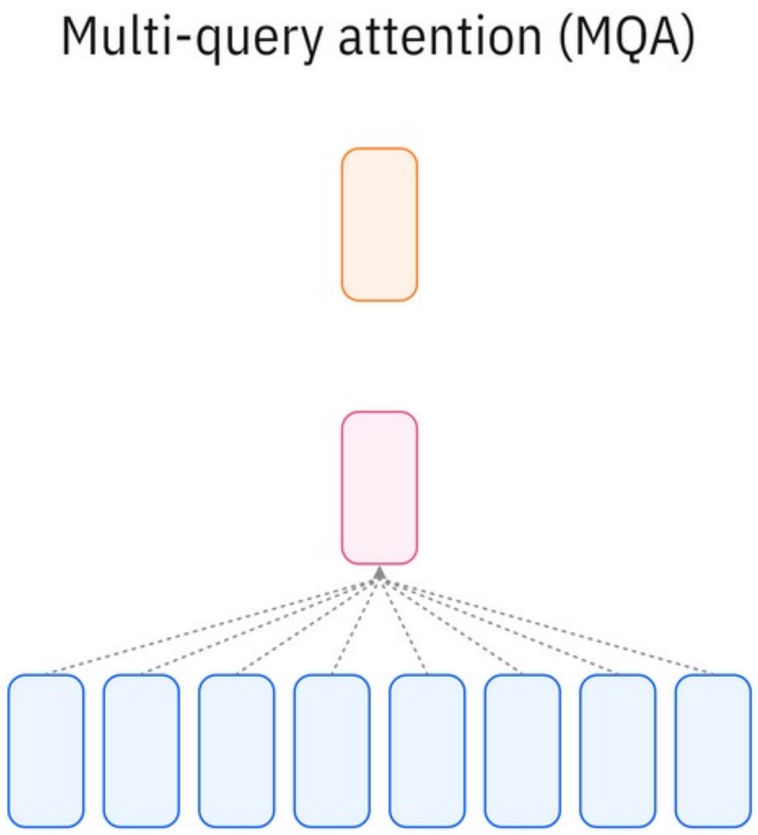
# Multi-Query Attention (MQA)

- MQA allows all query heads to share the same set of keys K and values V. Specifically, if we average all and from traditional MHA as follows:

$$\mathbf{K}^* = \frac{1}{H}\sum_{h=1}^{H}\mathbf{K}_h, \quad \mathbf{V}^* = \frac{1}{H}\sum_{h=1}^{H}\mathbf{V}_h,$$

$$\mathrm{MQA}(\mathbf{Q}, \mathbf{K}^*, \mathbf{V}^*) = \mathrm{Concat}(\mathrm{head}_1, \ldots, \mathrm{head}_H)\, W_O$$

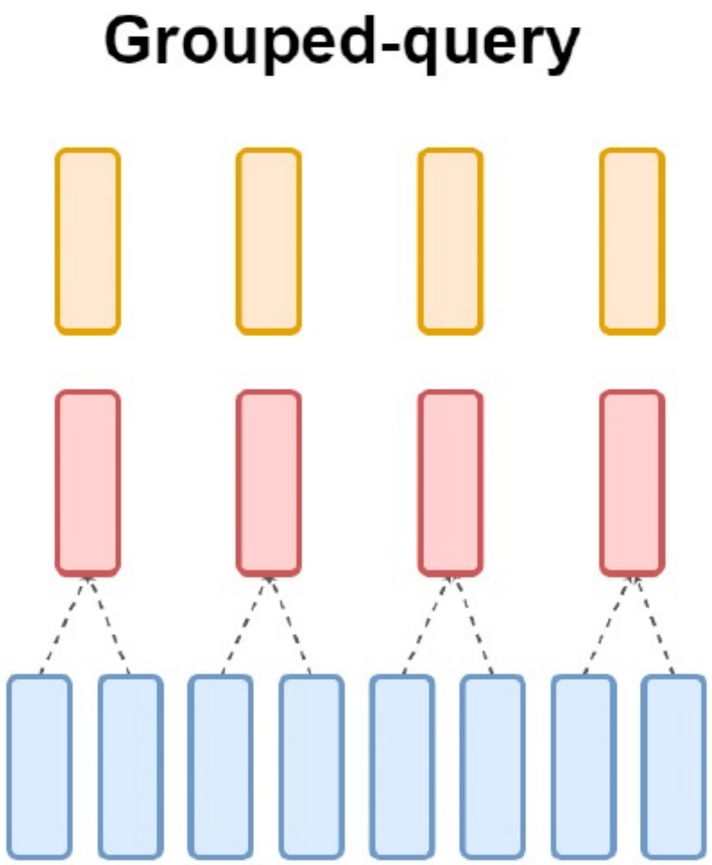
- MQA inference is faster but may limit the model's expressive capacity and performance in certain scenarios



# Grouped-Query Attention (GQA)

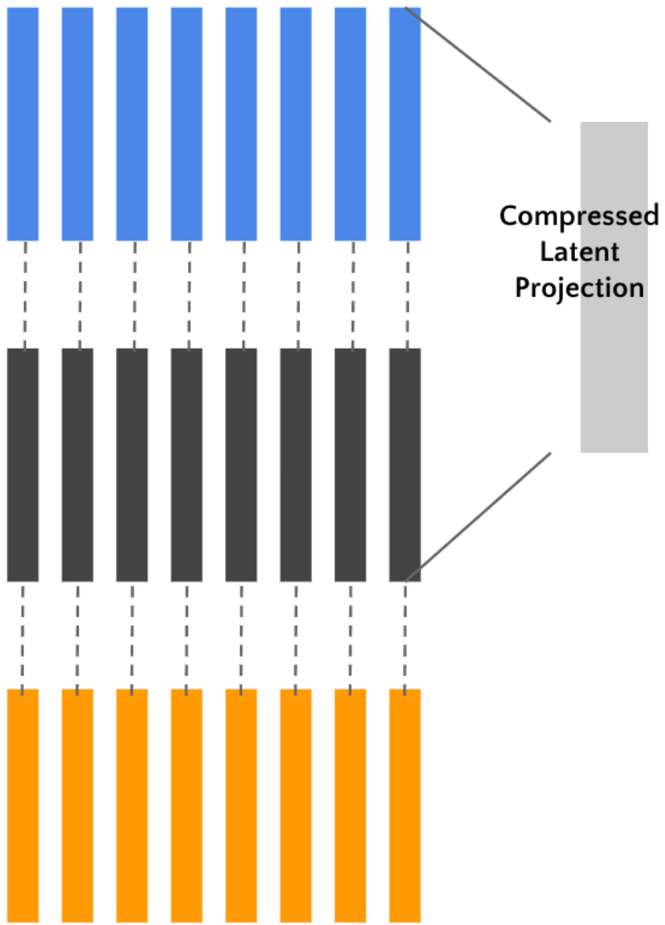
- GQA divides query heads into multiple groups, allowing each group to share a set of K and V heads, thereby balancing inference speed and model performance

- Each group contains $\frac{H}{G}$ query heads and shares one set of K and V heads



# Multi Head Latent Attention (MLA)

- MLA incorporates latent representations into the attention mechanism to reduce computational complexity and improve contextual representation




Multi Head Latent Attention (MLA)

# Multi Head Latent Attention (MLA)

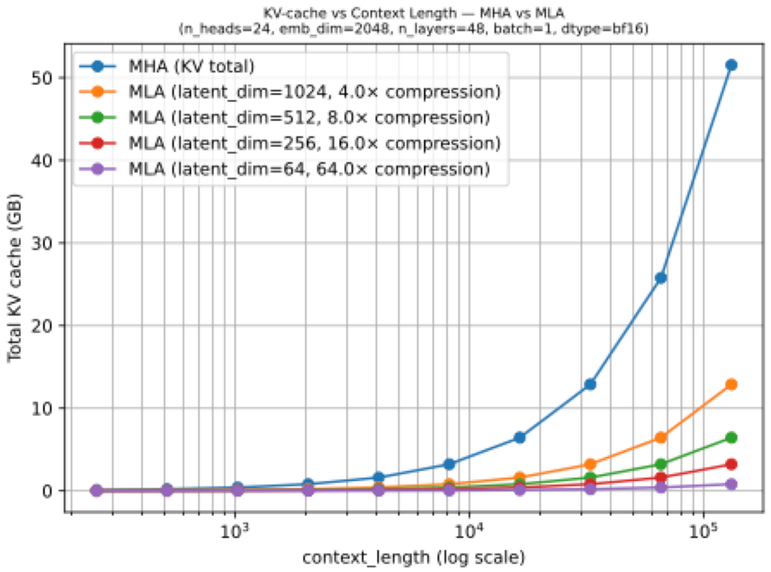
- MLA incorporates latent representations into the attention mechanism to reduce computational complexity and improve contextual representation

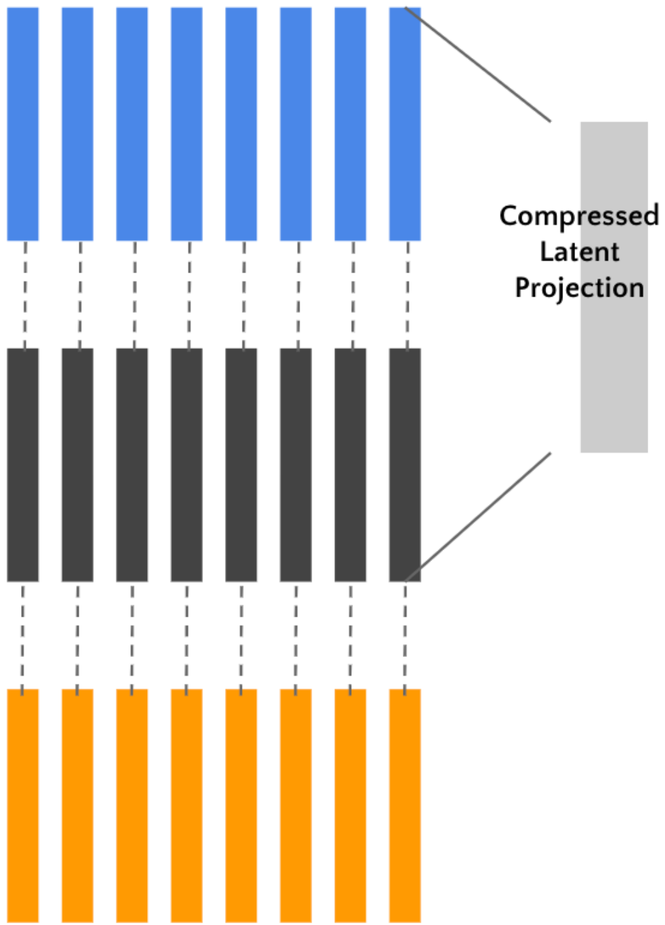
- Pros:

  > Reduces memory and computation cost compared to MHA;

  >Performs well on long sequences;

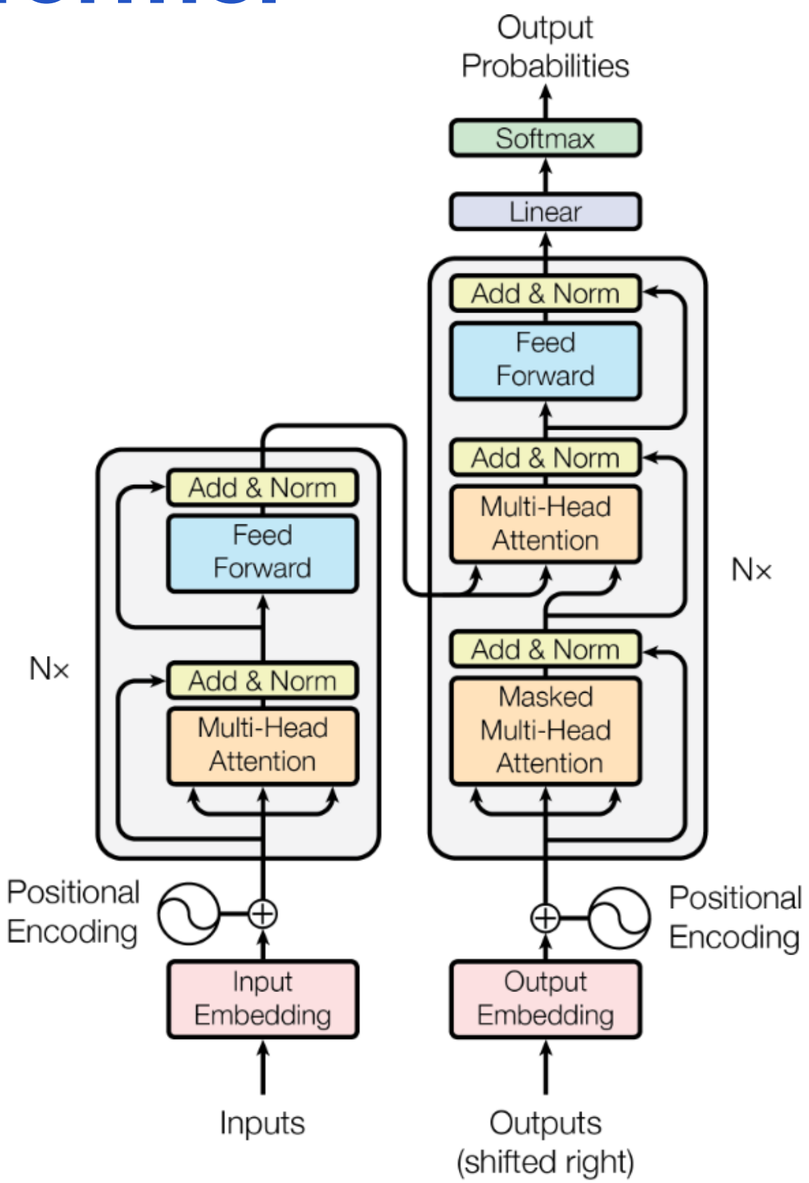
  > Latent embeddings provide higher abstraction



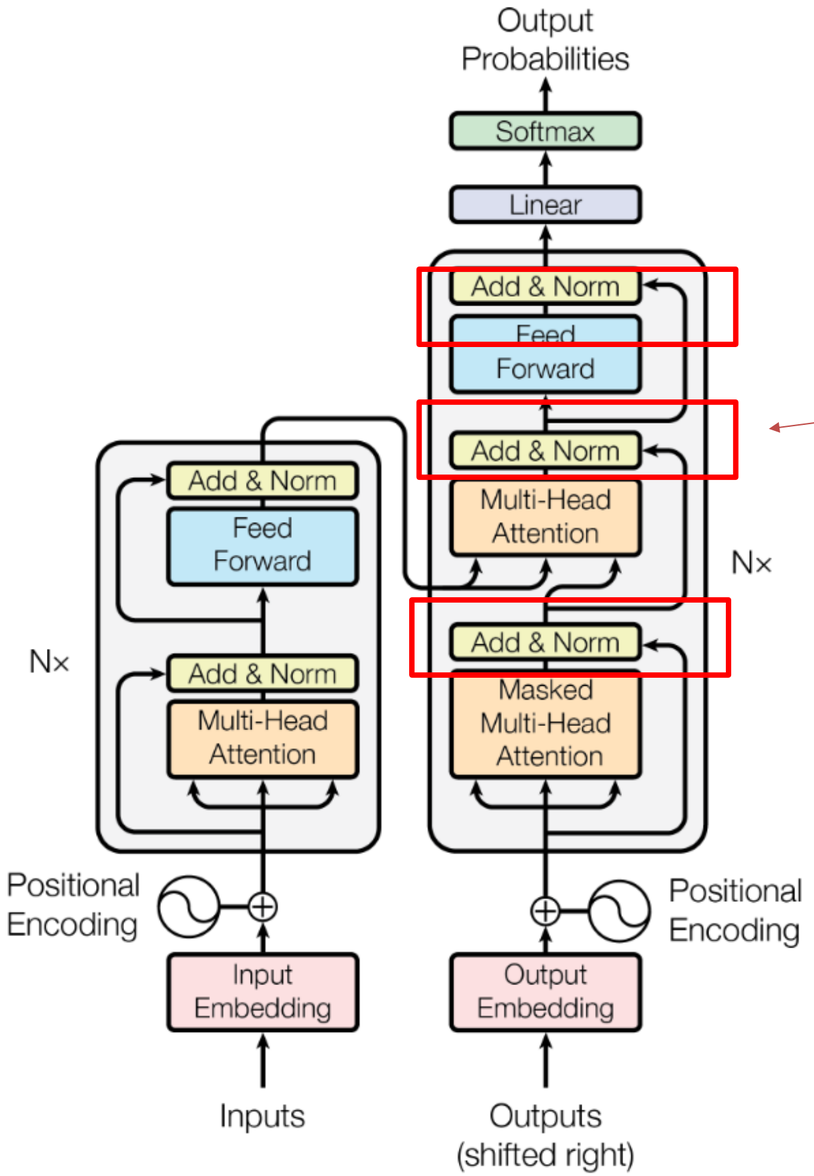




Multi Head Latent Attention (MLA)

# Vanilla Transformer



# Vanilla Transformer: Norms

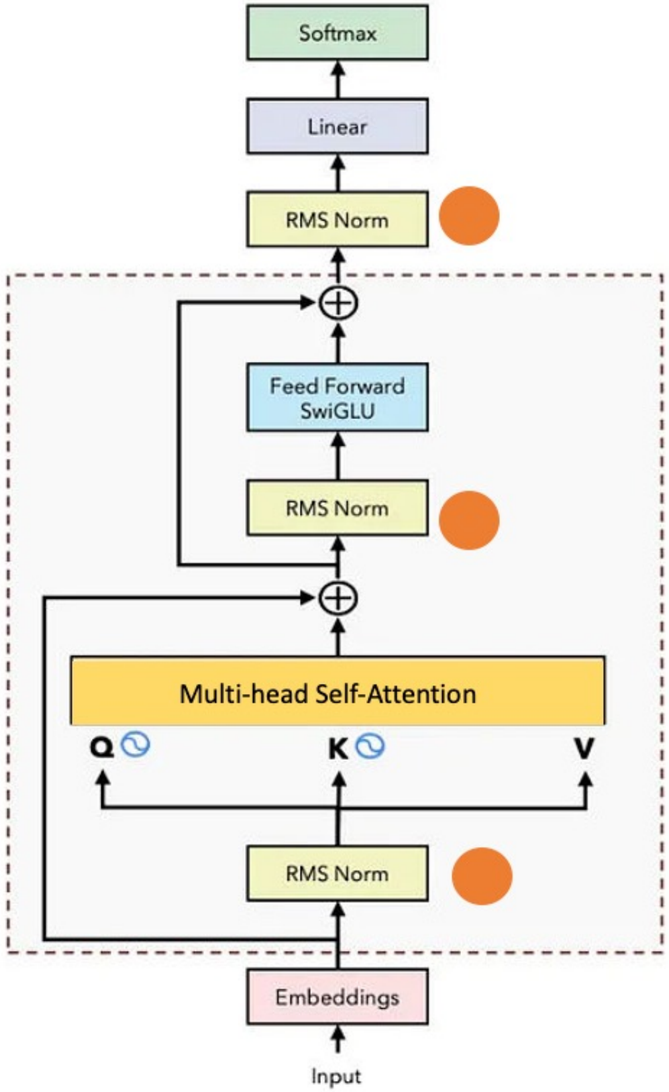



🤔 Could we do better?

# Pre-normalization of Input

- Given an $x \in R^n$, FFN projects it into y $\in R^n$

$$a_i = \sum_{j=1}^{m} w_{ij} x_j, \quad y_i = f(a_i + b_i),$$



# Pre-normalization of Input

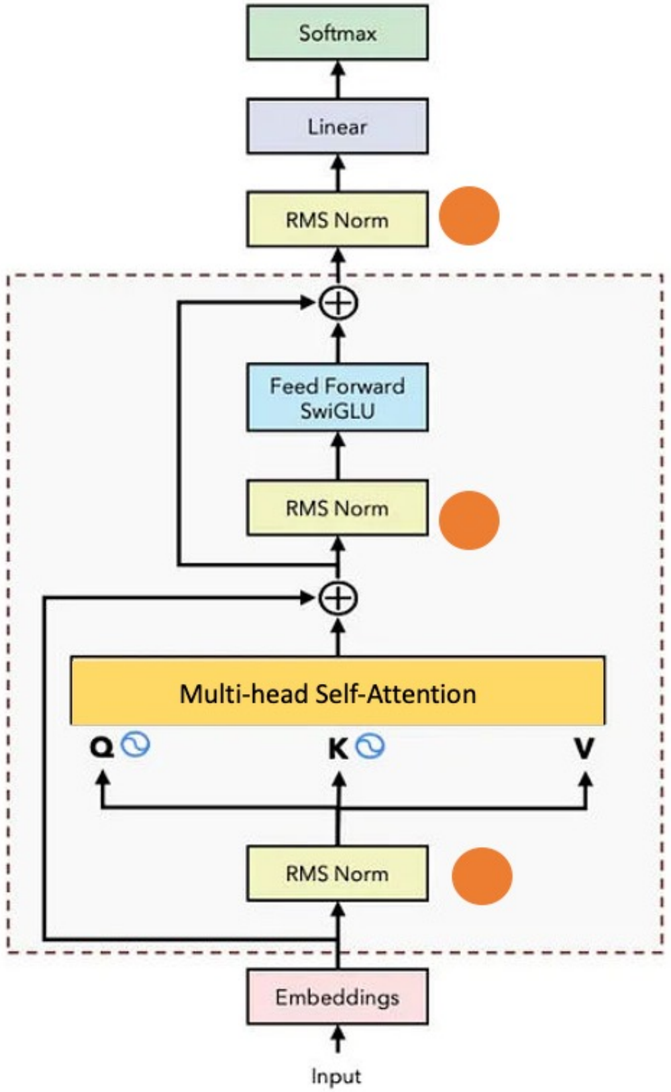
- Given an $x \in R^n$, FFN projects it into y $\in R^n$

$$a_i = \sum_{j=1}^{m} w_{ij} x_j, \quad y_i = f(a_i + b_i),$$

- Layer Norm (from Vanilla):

$$\bar{a}_i = \frac{a_i - \mu}{\sigma} g_i, \quad y_i = f(\bar{a}_i + b_i),$$

$$\mu = \frac{1}{n} \sum_{i=1}^{n} a_i, \quad \sigma = \sqrt{\frac{1}{n} \sum_{i=1}^{n} (a_i - \mu)^2}.$$



# Pre-normalization of Input

- Given an $x \in R^n$, FFN projects it into y $\in R^n$

$$a_i = \sum_{j=1}^{m} w_{ij} x_j, \quad y_i = f(a_i + b_i),$$

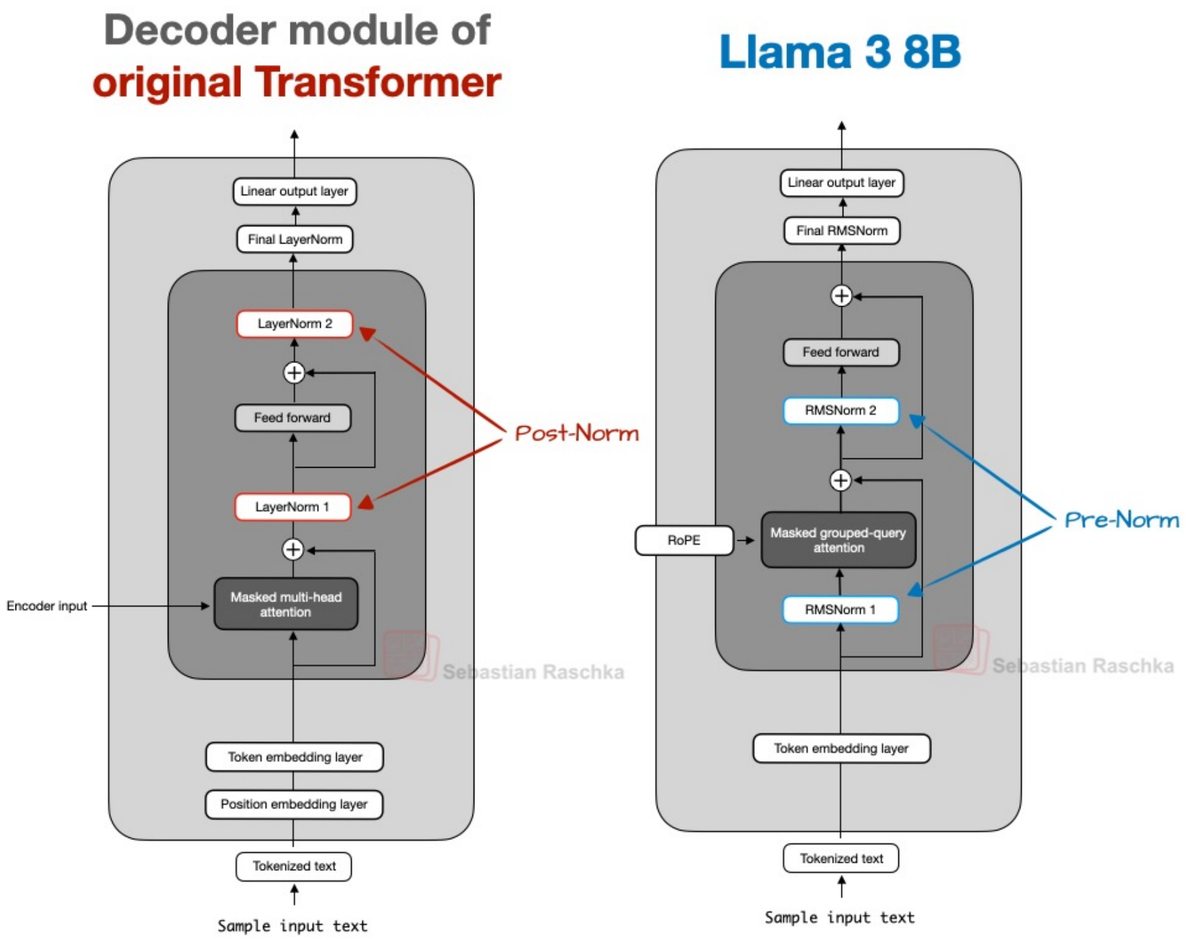
- Layer Norm (from Vanilla):

$$\bar{a}_i = \frac{a_i - \mu}{\sigma} g_i, \quad y_i = f(\bar{a}_i + b_i),$$

$$\mu = \frac{1}{n}\sum_{i=1}^{n} a_i, \quad \sigma = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(a_i - \mu)^2}.$$

- RMS Norm:

$$\bar{a}_i = \frac{a_i}{\mathrm{RMS}(\mathbf{a})} g_i, \quad \text{where } \mathrm{RMS}(\mathbf{a}) = \sqrt{\frac{1}{n}\sum_{i=1}^{n} a_i^2}.$$

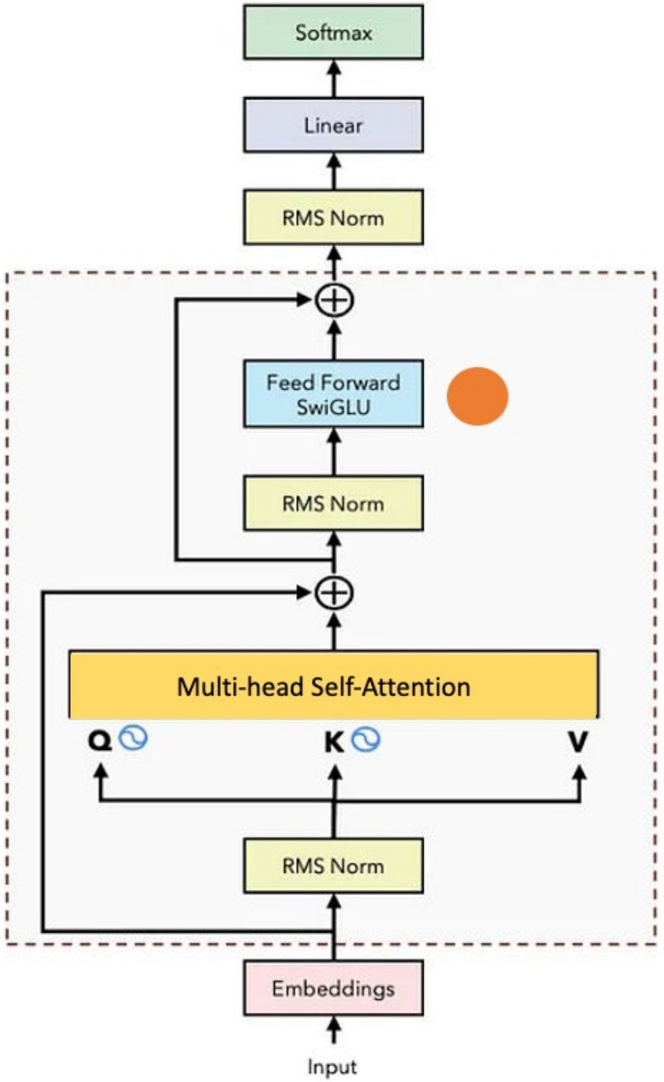


# SwiGLU

$$ReLU(x) = \max(0, x)$$

$$GLU(x) = x * \sigma(Wx + b)$$

$$SwiGLU(x) = x\sigma(\beta x) + (1 - \sigma(\beta * x)) * (Wx + b)$$

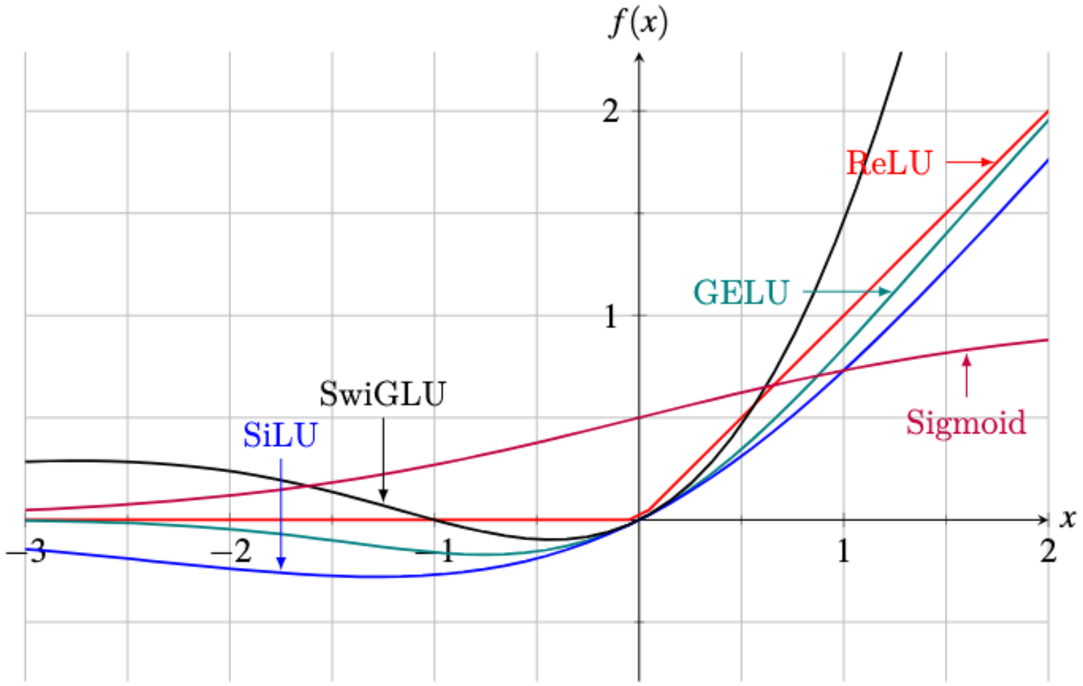


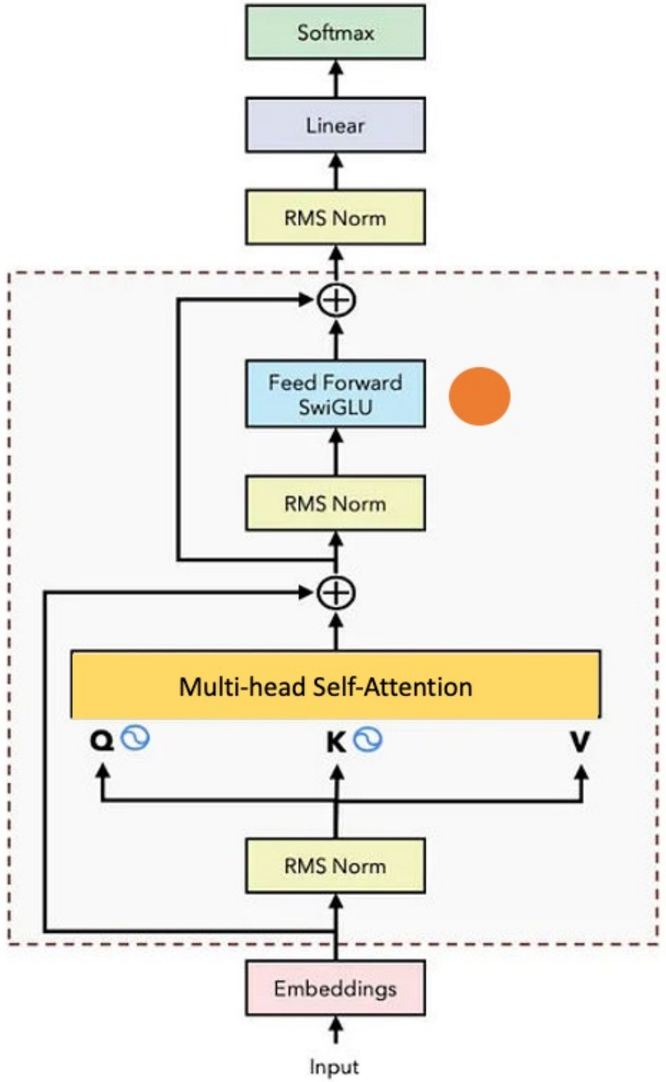
# SwiGLU

$$ReLU(x) = \max(0, x)$$

$$GLU(x) = x * \sigma(Wx + b)$$

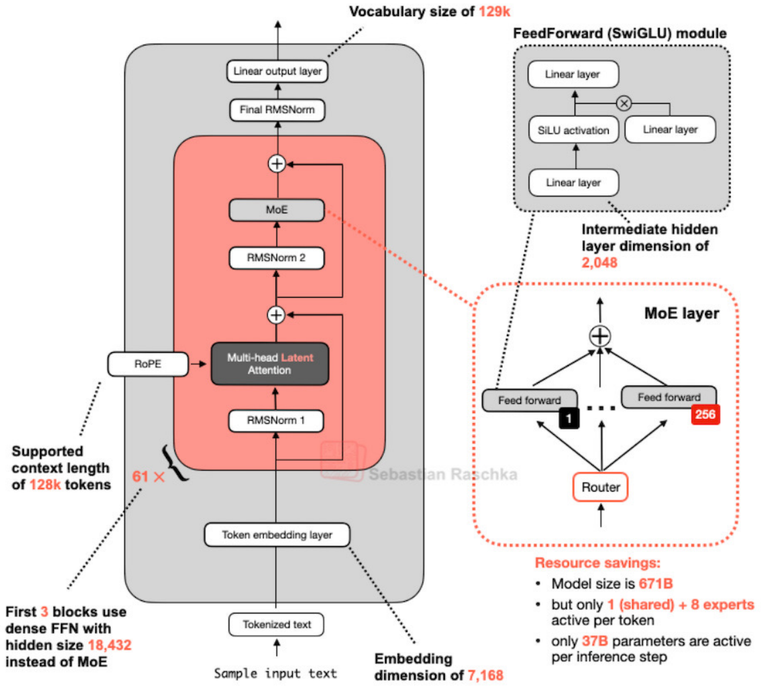
$$SwiGLU(x) = x\sigma(\beta x) + (1 - \sigma(\beta * x)) * (Wx + b)$$



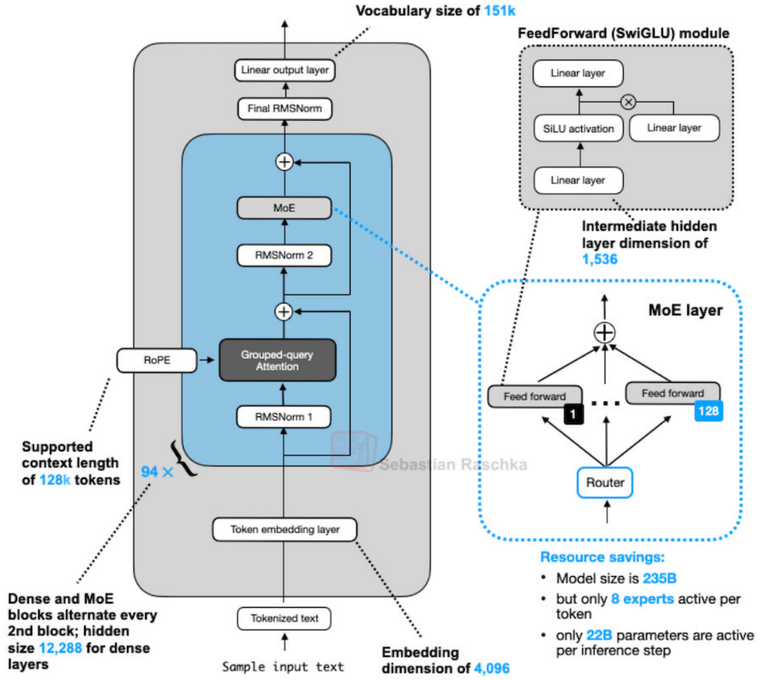



# Modern Architectures

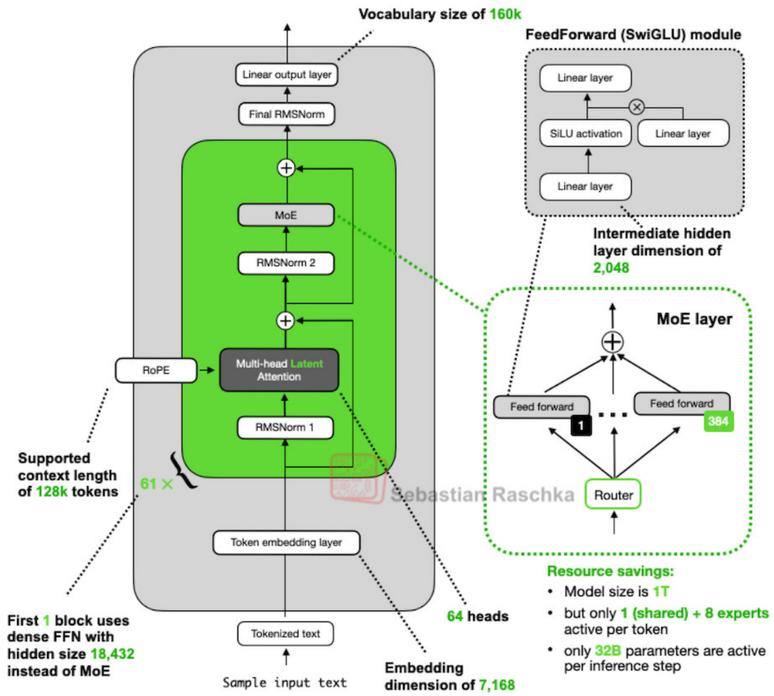
## DeepSeek V3 (671B)




## Qwen3 235B-A22B




## Kimi K2 (1 trillion)



# Recap

- The original Transformer remained the foundation, but modern LLMs changed many internal components
- RoPE and ALiBi improved how models encode position and extrapolate to longer inputs
- KV-cache made autoregressive inference much more practical
- MQA, GQA, and MLA reduced the KV bottleneck in different ways
- RMSNorm and SwiGLU became standard due to producing large gains in stability and efficiency

Next:

- Training large models